

2026 edition

Deep Learning for Music Analysis and Generation

Fundamentals for Musical Audio



Yi-Hsuan Yang Ph.D.
yhyangtw@ntu.edu.tw

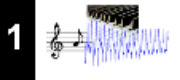
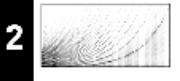
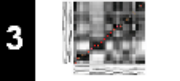
Outline

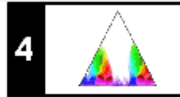
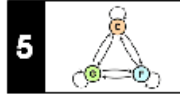
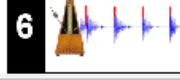
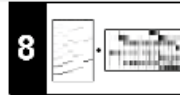
- Time and frequency representations for music
- Math in STFT

FMP Notebook

<https://www.audiolabs-erlangen.de/resources/MIR/FMP/C1/C1.html>

<https://www.audiolabs-erlangen.de/resources/MIR/FMP/C2/C2.html>

Part	Title	Notions, Techniques & Algorithms	HTML	IPYNB
	Basics	Basic information on Python, Jupyter notebooks, Anaconda package management system, Python environments, visualizations, and other topics	[html]	[ipynb]
	Overview	Overview of the notebooks (https://www.audiolabs-erlangen.de/FMP)	[html]	[ipynb]
	Music Representations	Music notation, MIDI, audio signal, waveform, pitch, loudness, timbre	[html]	[ipynb]
	Fourier Analysis of Signals	Discrete/analog signal, sinusoid, exponential, Fourier transform, Fourier representation, DFT, FFT, STFT	[html]	[ipynb]
	Music Synchronization	Chroma feature, dynamic programming, dynamic time warping (DTW), alignment, user interface	[html]	[ipynb]

Part	Title	Notions, Techniques & Algorithms	HTML	IPYNB
	Music Structure Analysis	Similarity matrix, repetition, thumbnail, homogeneity, novelty, evaluation, precision, recall, F-measure, visualization, scape plot	[html]	[ipynb]
	Chord Recognition	Harmony, music theory, chords, scales, templates, hidden Markov model (HMM), evaluation	[html]	[ipynb]
	Tempo and Beat Tracking	Onset, novelty, tempo, tempogram, beat, periodicity, Fourier analysis, autocorrelation	[html]	[ipynb]
	Content-Based Audio Retrieval	Identification, fingerprint, indexing, inverted list, matching, version, cover song	[html]	[ipynb]
	Musically Informed Audio Decomposition	Harmonic/percussive separation, signal reconstruction, instantaneous frequency, fundamental frequency (F0), trajectory, nonnegative matrix factorization (NMF)	[html]	[ipynb]

Outline

- **Time and frequency representations for music**
- Math in STFT

Audio Waveforms

https://www.audiolabs-erlangen.de/resources/MIR/FMP/C1/C1S3_Waveform.html

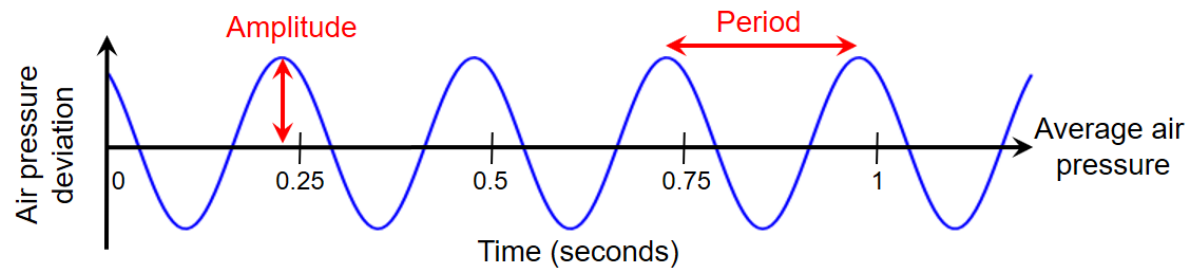
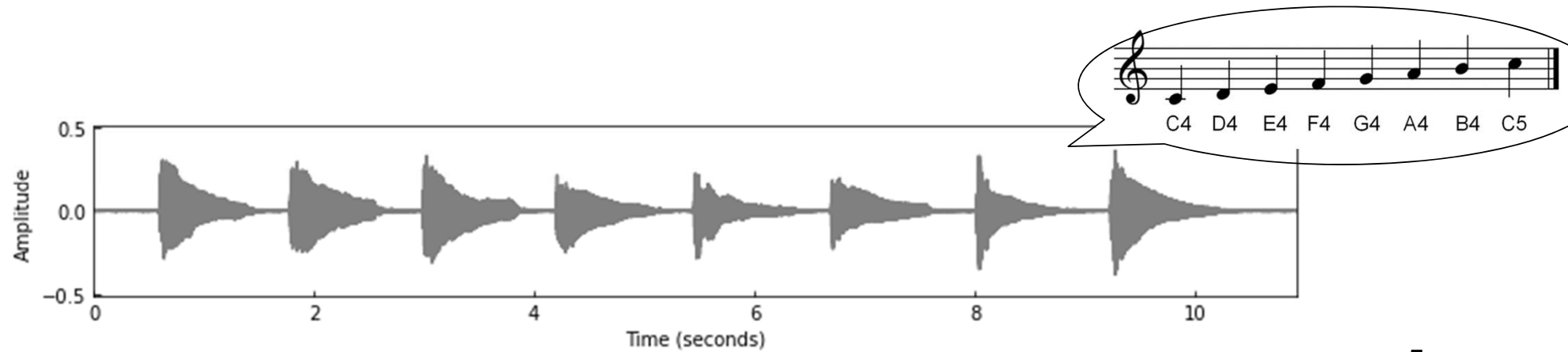


Figure 1.19 from [Müller, FMP, Springer 2015]



What Do We Perceive in Musical Audio?

- Loudness
- Spatial information
- Timbre / instrument / singer identity
- Pitch / melody
- Chord / harmony
- Tempo / rhythm
- Style
- Emotion
- Others (e.g., “The power of music compounds over time through repeated listening. Culture builds around shared experiences.”)

Dynamics, Intensity, and Loudness

https://www.audiolabs-erlangen.de/resources/MIR/FMP/C1/C1S3_Dynamics.html

- **Intensity:** a physical property
 - Other names: power (dB); energy
 - **Peak** vs. root mean square (**RMS**)
 - Dynamic range: dB max/min difference
- **Loudness:** a perceptual property
 - **LUFS:** frequency-weighted average
 - volume: user control of loudness

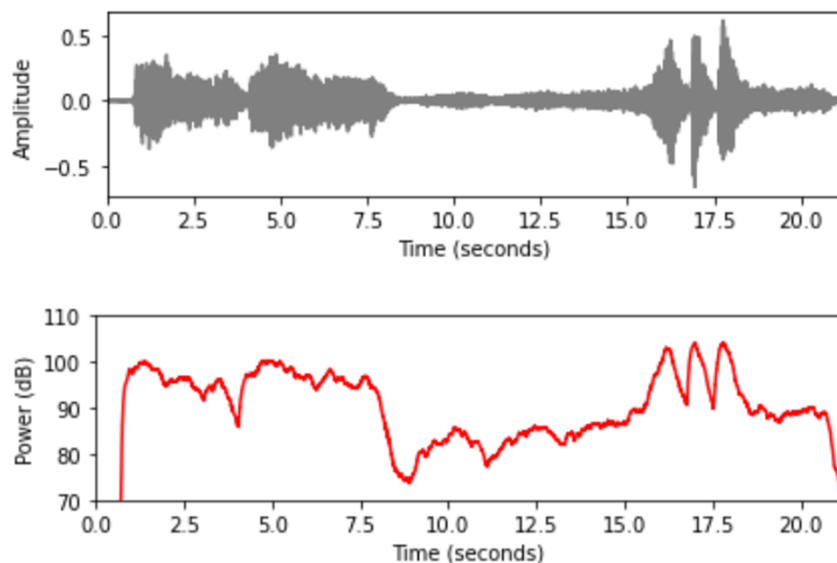
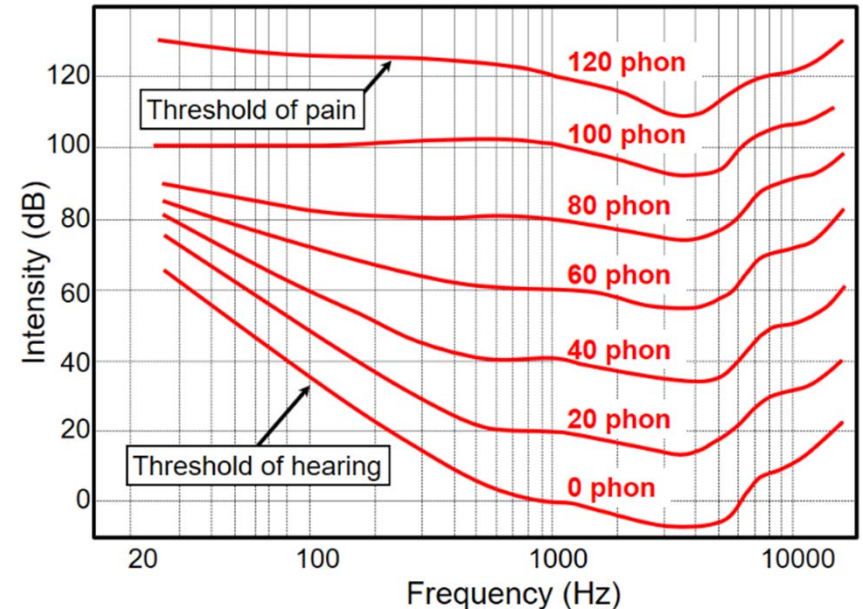


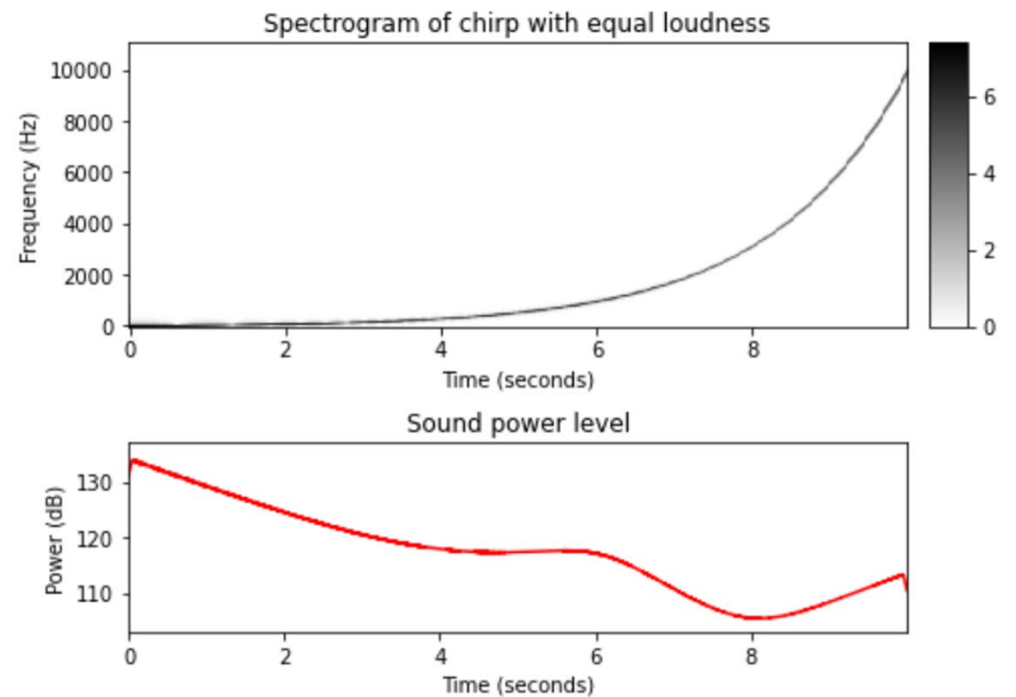
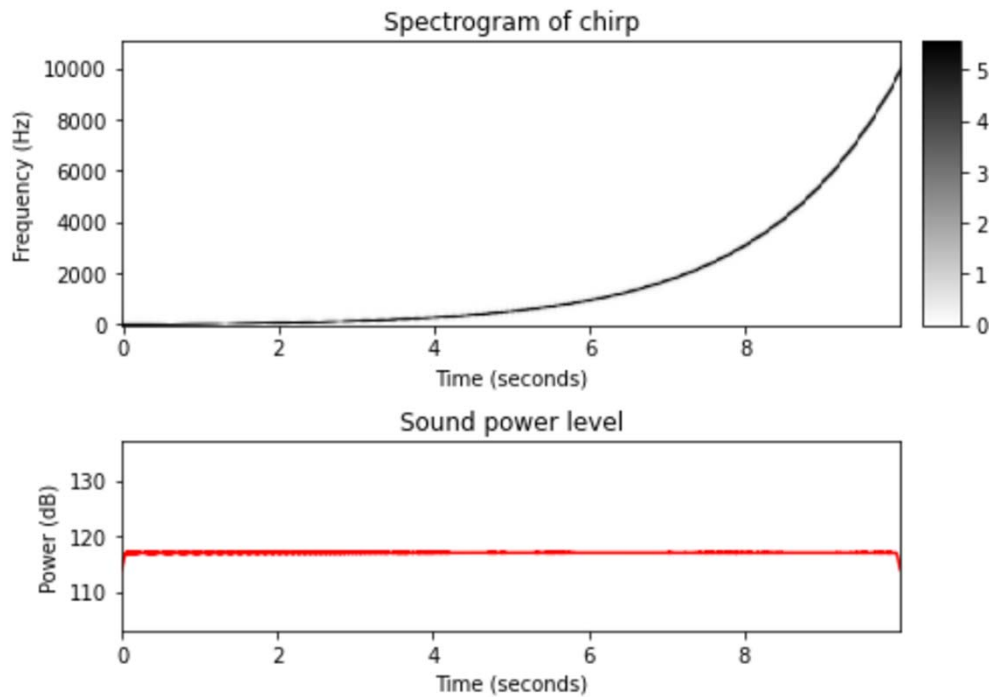
Figure 1.21 from [Müller, FMP, Springer 2015]



Dynamics, Intensity, and Loudness

https://www.audiolabs-erlangen.de/resources/MIR/FMP/C1/C1S3_Dynamics.html

- Chirp signal with equal power
- Chirp signal with equal loudness



Dynamics, Intensity, and Loudness

https://www.audiolabs-erlangen.de/resources/MIR/FMP/C1/C1S3_Dynamics.html

- Mixing, mastering



- Audio normalization

– E.g., <https://github.com/csteinmetz1/pyloudnorm>

```
import soundfile as sf
import pyloudnorm as pyln

data, rate = sf.read("test.wav") # load audio

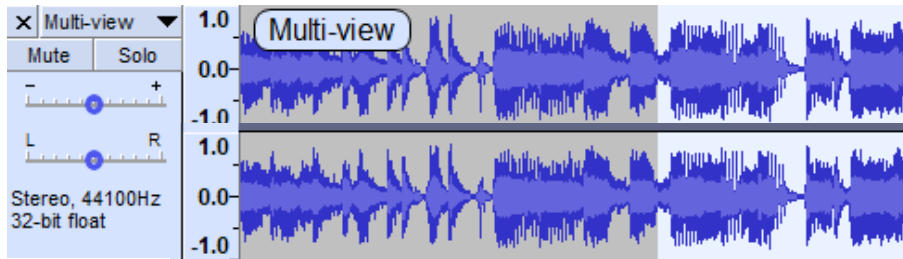
# peak normalize audio to -1 dB
peak_normalized_audio = pyln.normalize.peak(data, -1.0)

# measure the loudness first
meter = pyln.Meter(rate) # create BS.1770 meter
loudness = meter.integrated_loudness(data)

# loudness normalize audio to -12 dB LUFS
loudness_normalized_audio = pyln.normalize.loudness(data, loudness, -12.0)
```

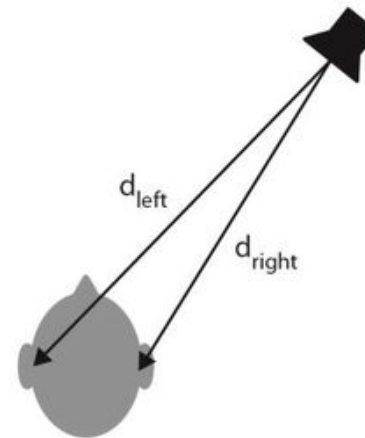
Spatial Information: Mono vs Stereo

- Monaural vs. binaural

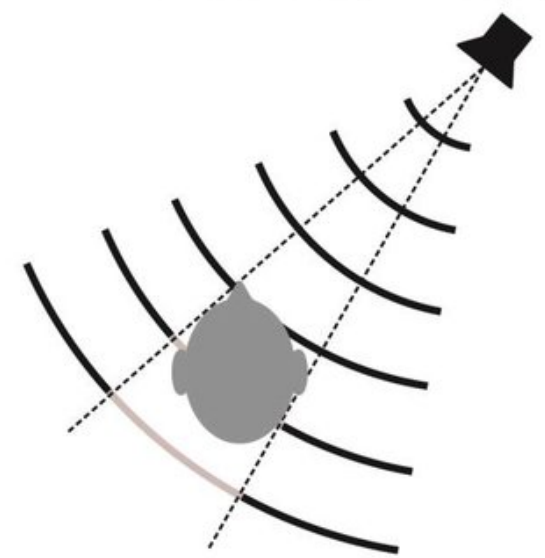


- Our auditory system constructs a **3D** mental model of a soundscape
- **Localization** of sound objects
 - Azimuth, elevation, distance
- Perception of **room acoustics**

a) Binaural localization cue: interaural time difference (ITD)



b) Binaural localization cue: interaural intensity difference (IID)



Source: https://www.researchgate.net/figure/Binaural-and-monaural-cues-used-for-sound-localization-a-For-sound-sources-off-the_fig1_299281975

torchaudio

<https://docs.pytorch.org/audio/stable/generated/torchaudio.load.html>

https://docs.pytorch.org/audio/2.0.1/tutorials/audio_io_tutorial.html

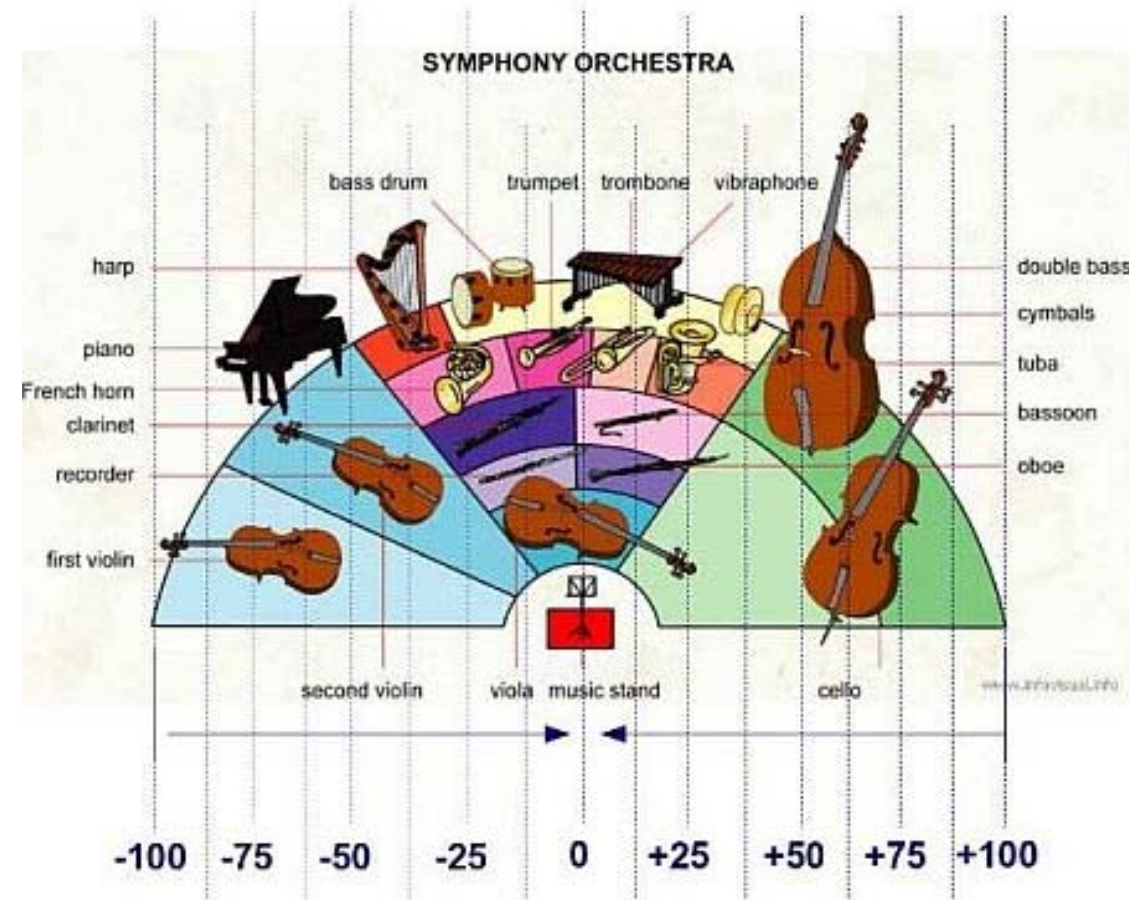
```
waveform, sample_rate = torchaudio.load(SAMPLE_WAV)
```

```
def plot_waveform(waveform, sample_rate):  
    waveform = waveform.numpy()  
    num_channels, num_frames = waveform.shape
```

Spatial Information



<https://www.renegadeproducer.com/audio-panning.html>



<https://www.audiorecording.me/symphony-orchestra-panning-and-reverb-settings.html>

Timbre: Different Instruments

https://www.audiolabs-erlangen.de/resources/MIR/FMP/C1/C1S3_Timbre.html

- **Timbre** (tone color; tone quality)
 - Allow us to distinguish the musical tone (i.e., a single musical note) of different instruments, even if the tone is played at the same **pitch** and with the same **loudness**
 - Subjective & it's hard to describe
 - May be described with words such as *bright*, *dark*, *warm*, and *harsh*

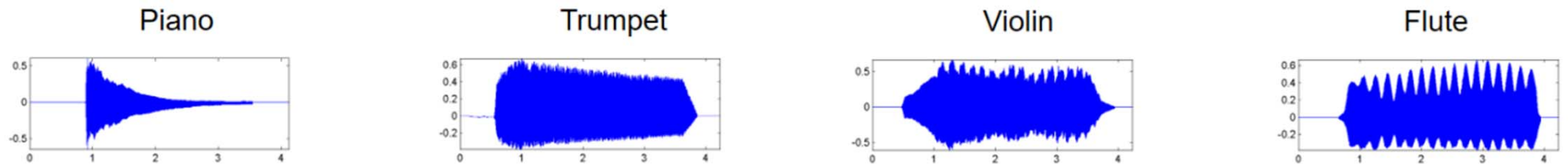


Figure 1.23 from [Müller, FMP, Springer 2015]

▶ 0:04 / 0:04 — 🔊

▶ 0:00 / 0:04 — 🔊

▶ 0:00 / 0:04 — 🔊

▶ 0:00 / 0:04 — 🔊

Timbre: Temporal Aspect

- **ADSR**: attack (A), decay (D), sustain (S), and release (R)
 - Hard to analyze when there are concurrent notes

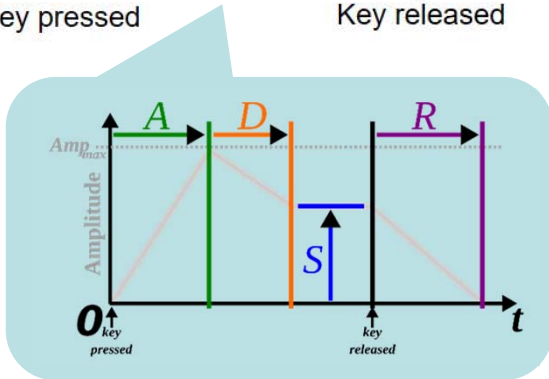
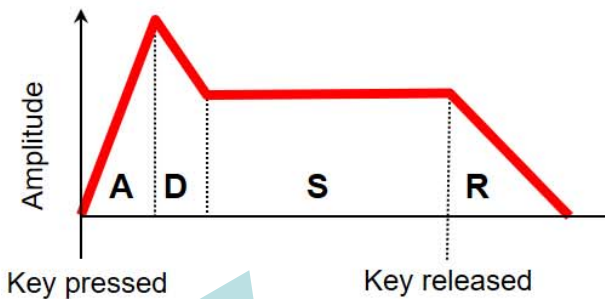
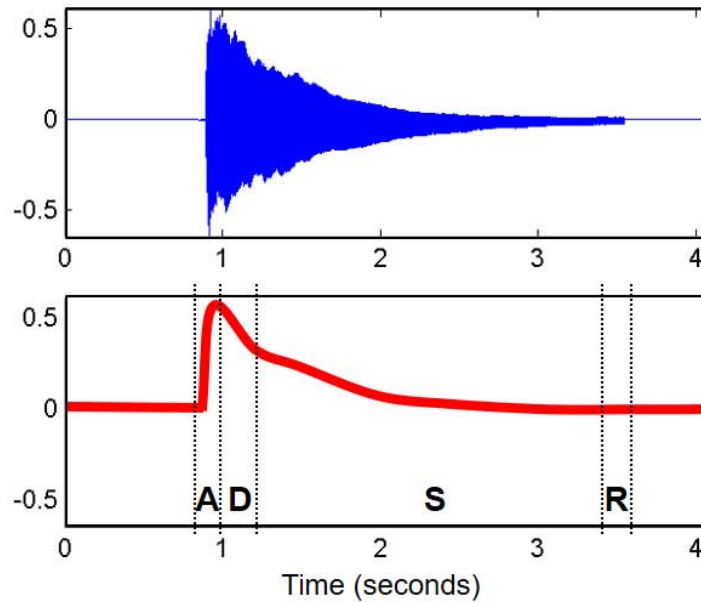


Figure from:
[https://en.wikipedia.org/wiki/Envelope_\(music\)](https://en.wikipedia.org/wiki/Envelope_(music))

Piano sound



Violin sound

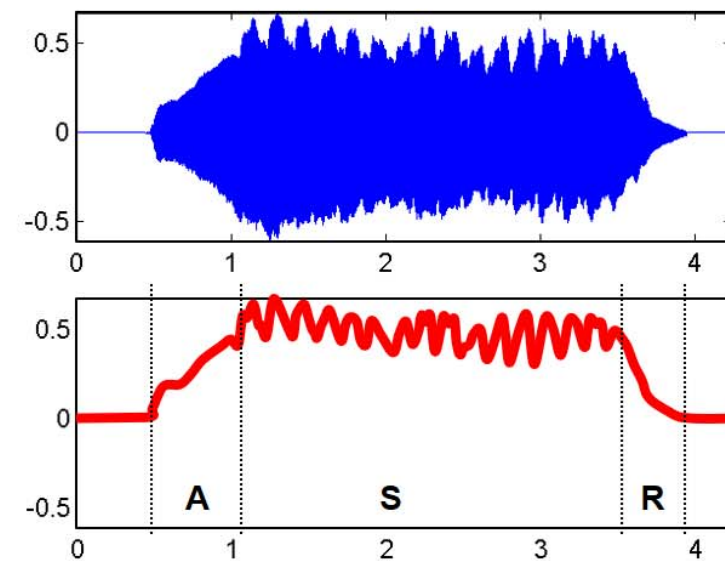


Figure 1.22b and Figure 1.23
from [Müller, FMP, Springer 2015]

Notes and Pitches

(Assuming A4=440 Hz; but there are *other tunings*)

<https://newt.phys.unsw.edu.au/jw/notes.html>

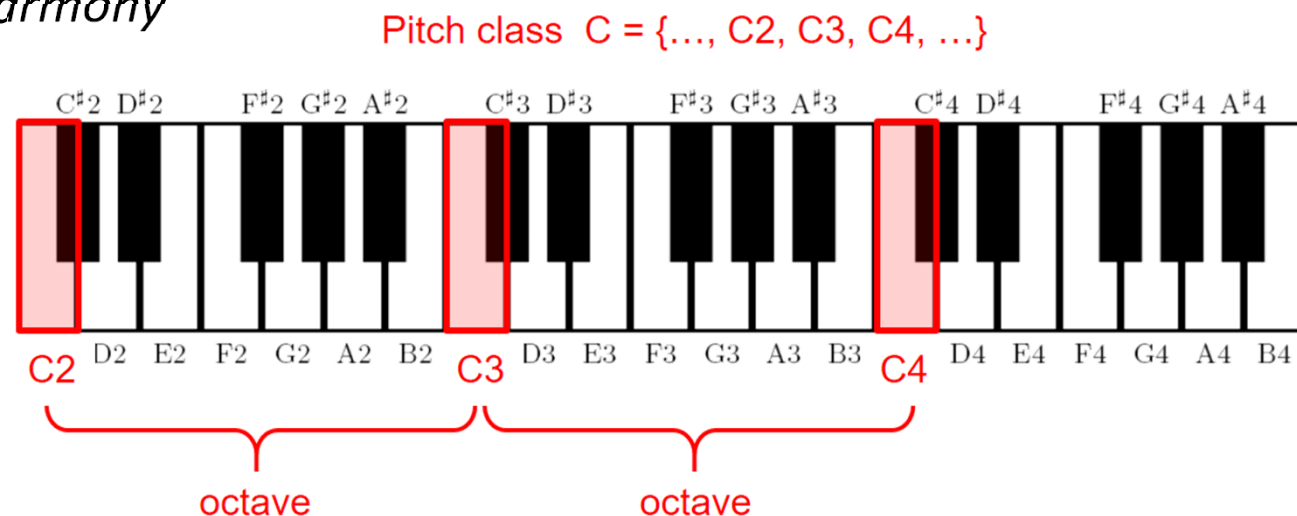
Note name	Keyboard	Frequency Hz		Frequency Hz		Frequency Hz		Frequency Hz		Frequency Hz
A0		27.500		F3		174.61		D6		1174.7
B0		30.868	29.135	G3		196.00	185.00	E6		1318.5
C1		32.703		A3		220.00	207.65	F6		1396.9
D1		36.708	34.648	B3		246.94	233.08	G6		1568.0
E1		41.203	38.891	C4		261.63		A6		1760.0
F1		43.654		D4		293.67	277.18	B6		1975.5
G1		48.999	46.249	E4		329.63	311.13	C7		2093.0
A1		55.000	51.913	F4		349.23		D7		2349.3
B1		61.735	58.270	G4		392.00	369.99	E7		2637.0
C2		65.406		A4		440.00	415.30	F7		2793.0
D2		73.416	69.296	B4		493.88	466.16	G7		3136.0
E2		82.407	77.782	C5		523.25		A7		3520.0
F2		87.307		D5		587.33	554.37	B7		3951.1
G2		97.999	92.499	E5		659.26	622.25	C8		4186.0
A2		110.00	103.83	F5		698.46				
B2		123.47	116.54	G5		783.99	739.99			
C3		130.81		A5		880.00	830.61			
D3		146.83	138.59	B5		987.77	932.33			
E3		164.81	155.56	C6		1046.5				

Notes and Pitches

https://www.audiolabs-erlangen.de/resources/MIR/FMP/C1/C1S1_MusicalNotesPitches.html

- **Pitch Class**

- Two notes with *fundamental frequencies* in a ratio equal to *any power of two* (e.g., half, twice, or four times) are perceived as very **similar**
- All notes with this kind of relation can be grouped under the same *pitch class*
- Related to *chords/harmony*



MIDI Note Numbers

https://en.wikipedia.org/wiki/MIDI_tuning_standard

$$f = 2^{(d-69)/12} \cdot 440 \text{ Hz} \quad d = 69 + 12 \log_2 \left(\frac{f}{440 \text{ Hz}} \right)$$

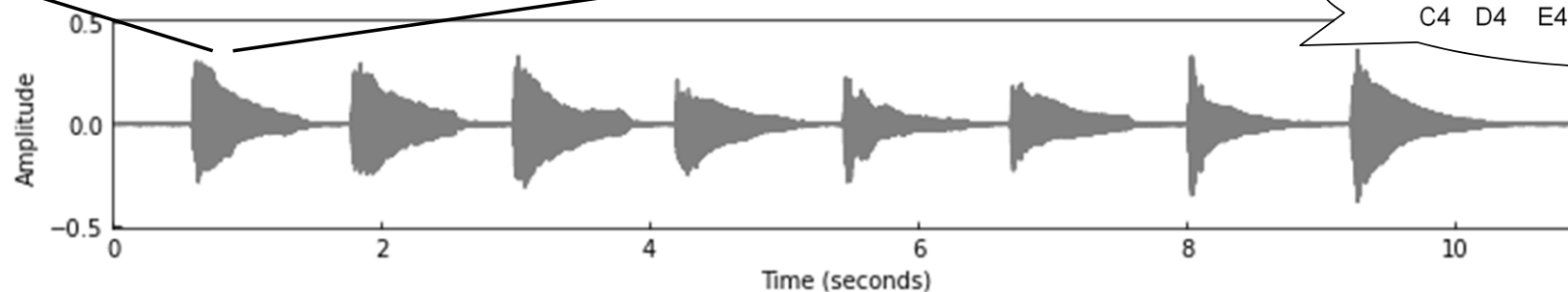
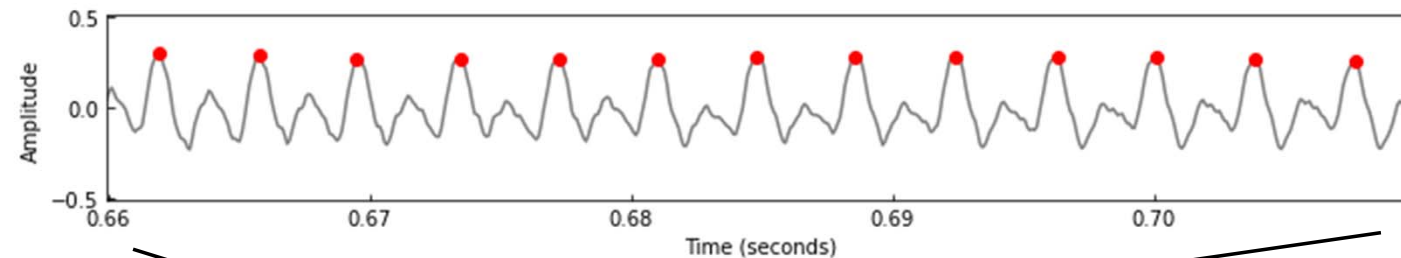
MIDI number	Note name	Keyboard	Frequency Hz	
21	A0		27.500	
23	B0		30.868	29.135
24	C1		32.703	
26	D1		36.708	34.648
28	E1		41.203	38.891
29	F1		43.654	
30	G1		48.999	46.249
31	A1		51.913	
32	B1		55.000	51.913
33	C2		58.270	
34	D2		61.735	58.270
35	E2		65.406	
36	F2		69.296	65.406
37	G2		73.416	69.296
38	A2		77.782	
39	B2		82.407	77.782
40	C3		87.307	
41	D3		92.499	87.307
42	E3		97.999	92.499
43	F3		103.83	
44	G3		110.00	103.83
45	A3		116.54	110.00
46	B3			116.54
55	C4		220.00	207.65
56	D4		246.94	233.08
57	E4		261.63	
58	F4		277.18	261.63
59	G4		293.67	277.18
60	A4		311.13	293.67
61	B4		329.63	311.13
62	C5		349.23	
63	D5		369.99	349.23
64	E5		392.00	369.99
65	F5		415.30	392.00
66	G5		440.00	415.30
67	A5		466.16	440.00
68	B5		495.88	466.16
69	C6		523.25	
70	D6		554.37	523.25
71	E6		587.33	554.37
72	F6		622.25	587.33
73	G6		659.26	622.25
74	A6		698.46	
75	B6		739.99	698.46
76	C7		783.99	739.99
77	D7		830.61	783.99
78	E7		880.00	830.61
79	F7		932.33	880.00
80	G7		987.77	932.33
81	A7		1046.5	987.77
82	B7			1046.5
83	C8			
84	D8			

Audio Waveforms (Cont')

https://www.audiolabs-erlangen.de/resources/MIR/FMP/C1/C1S3_Waveform.html

Zoom-in of the section between 0.66 and 0.71 seconds

- Highly repetitive
- 13 high-pressure (red) points $\rightarrow 20 \times 13 = 260$ Hz

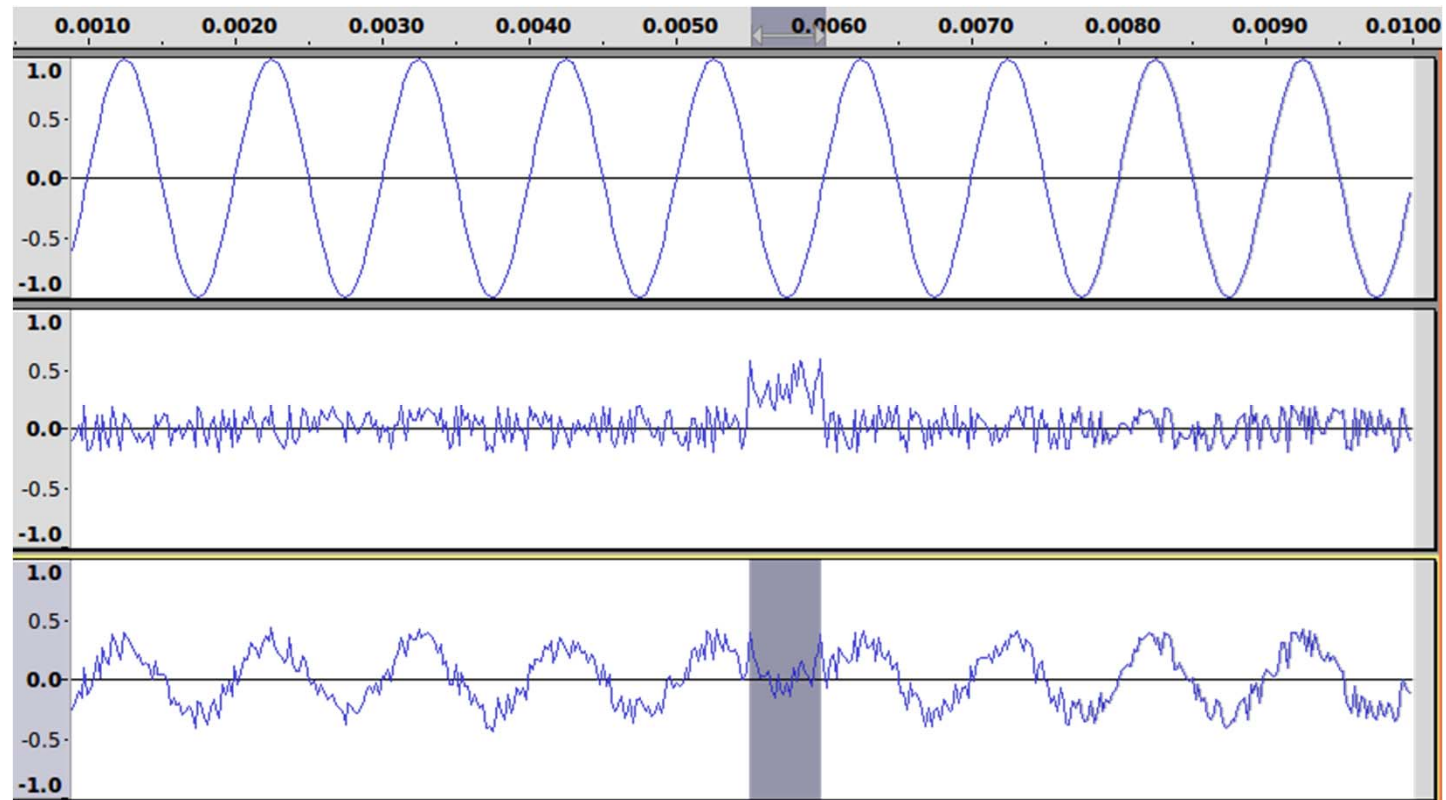


C4  261.63



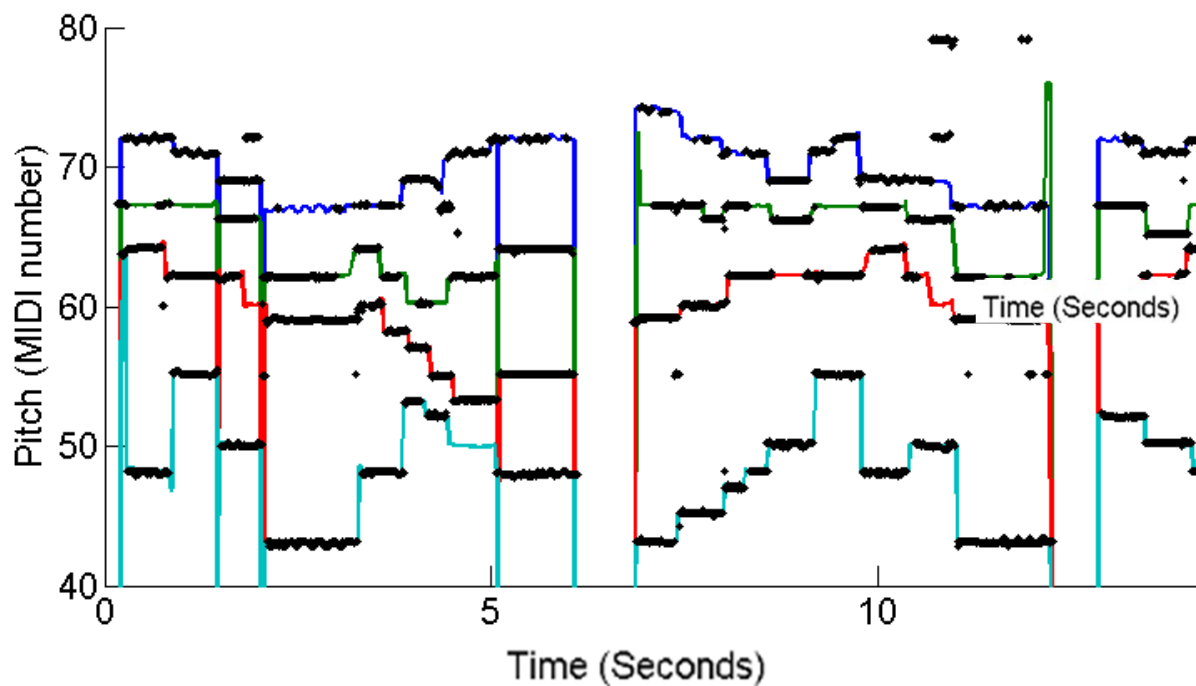
Temporal Characteristics : Pitched Sound or Not

- **Zero-crossing rate**
 - How often a signal changes its sign
 - **Noise**-like or not
 - Can be used in detecting the presence of vocals in speech signals
 - Cannot be used for estimating pitch due to noises and concurrent notes

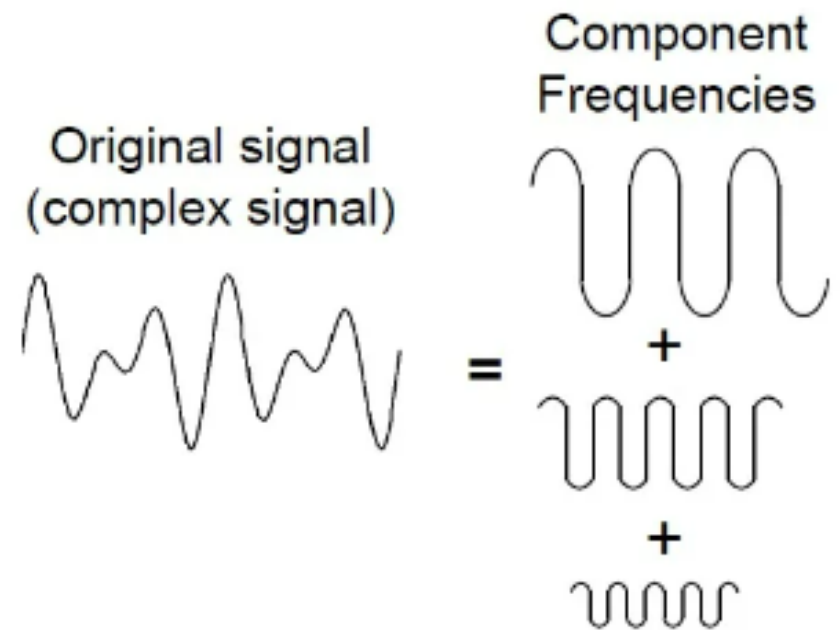


<https://sandilands.info/sgordon/impact-of-noise-on-sine-wave-compared-to-square-wave>

Multiple Concurrent Pitches



<https://labsites.rochester.edu/air/projects/multipitch/multipitch.html>



<https://www.allaboutcircuits.com/technical-articles/an-introduction-to-filters/>

Frequency-domain Analysis

- Main subject of “Signals and Systems”
- **Fourier Transform**

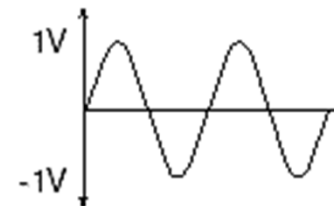
Description

Time Series

Fourier Expansion

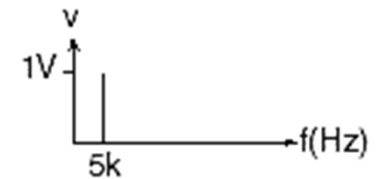
Power Spectrum

A pure 5kHz sine wave measuring 1 volt peak

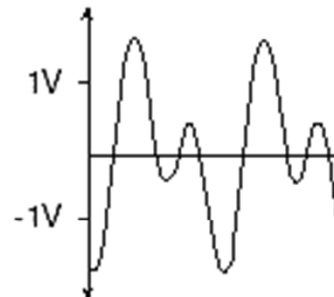


$$v(t) = 1\sin(\omega_1)t$$

$$\omega_1 = 2\pi(5\text{kHz})$$



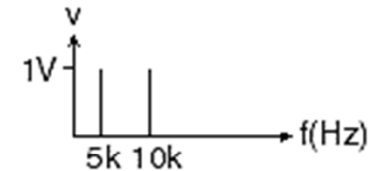
A pure 5kHz and 10kHz sine wave, each measuring 1 volt peak, added together



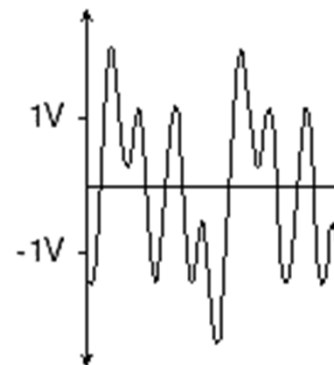
$$v(t) = 1\sin(\omega_1)t + 1\sin(\omega_2)t$$

$$\omega_1 = 2\pi(5\text{kHz})$$

$$\omega_2 = 2\pi(10\text{kHz})$$



A pure 5kHz, 10kHz, and 20kHz sine wave, each measuring 1 volt peak, added together

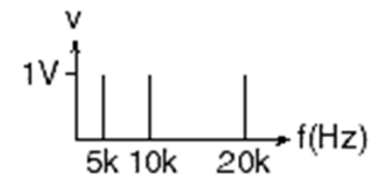


$$v(t) = 1\sin(\omega_1)t + 1\sin(\omega_2)t + 1\sin(\omega_3)t$$

$$\omega_1 = 2\pi(5\text{kHz})$$

$$\omega_2 = 2\pi(10\text{kHz})$$

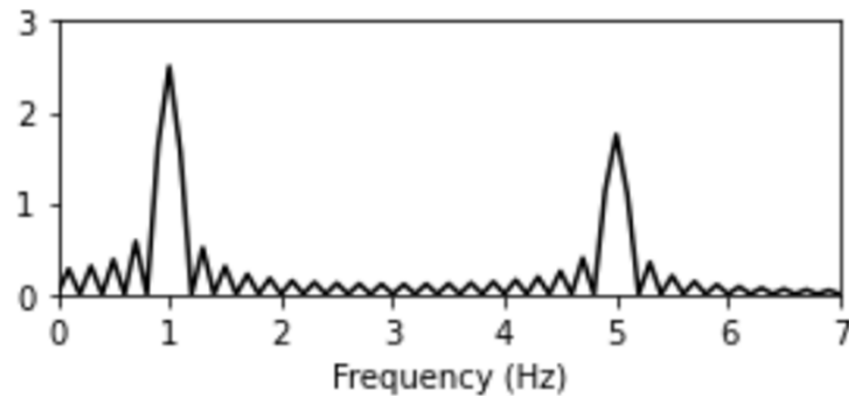
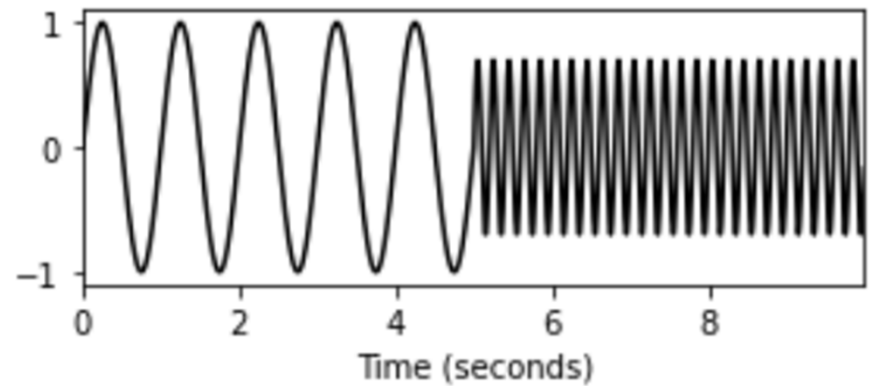
$$\omega_3 = 2\pi(20\text{kHz})$$



Frequency-domain Analysis

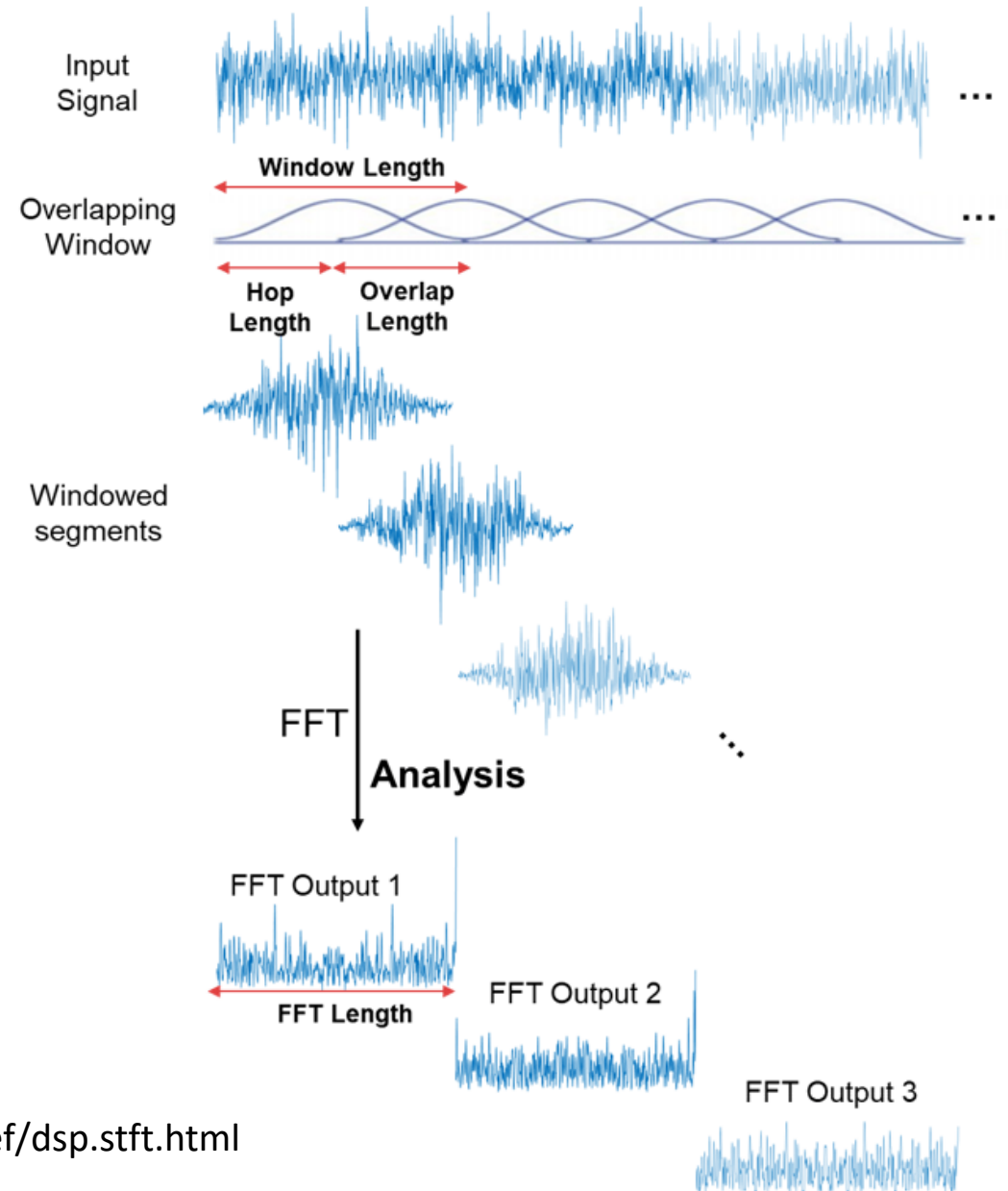
https://www.audiolabs-erlangen.de/resources/MIR/FMP/C2/C2_STFT-Basic.html

- **Fourier Transform cannot localize temporal events**



Frequency-domain Analysis

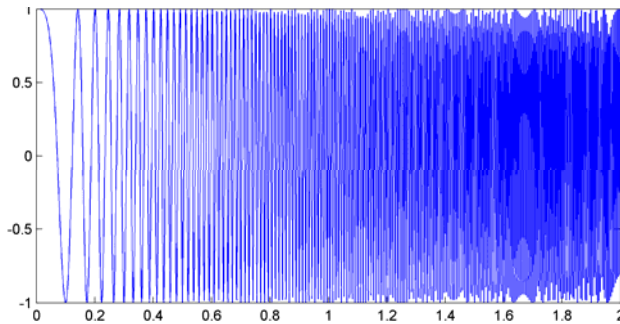
- Fourier Transform (FFT) cannot localize temporal events
- **Short-Time Fourier Transform (STFT)**
 - *Time-frequency representation* of signals
 - Slide **short windows** over the input signal and then compute the FFT locally for each windowed signal
- “**Window**”: a short chunk of time



<https://ww2.mathworks.cn/help/dsp/ref/dsp.stft.html>

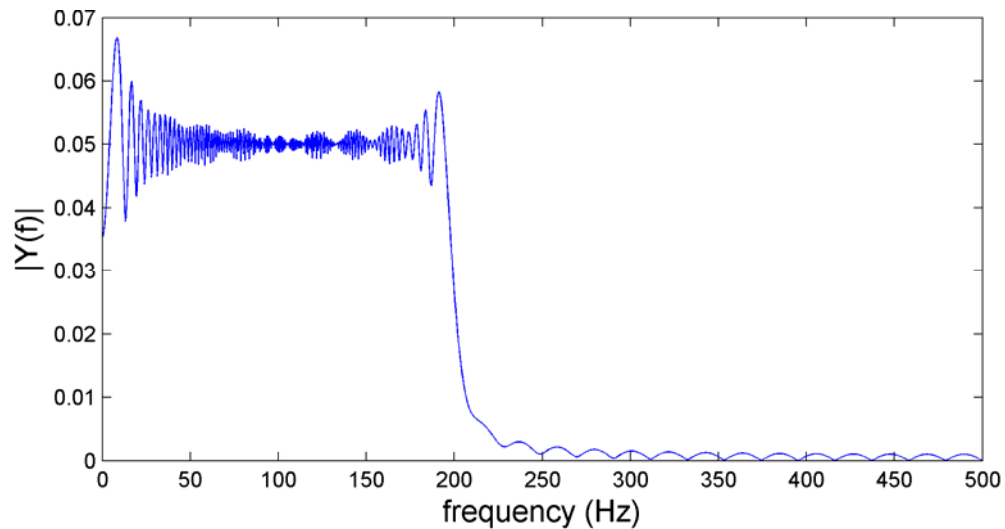
FFT vs STFT

Time-domain **waveform**

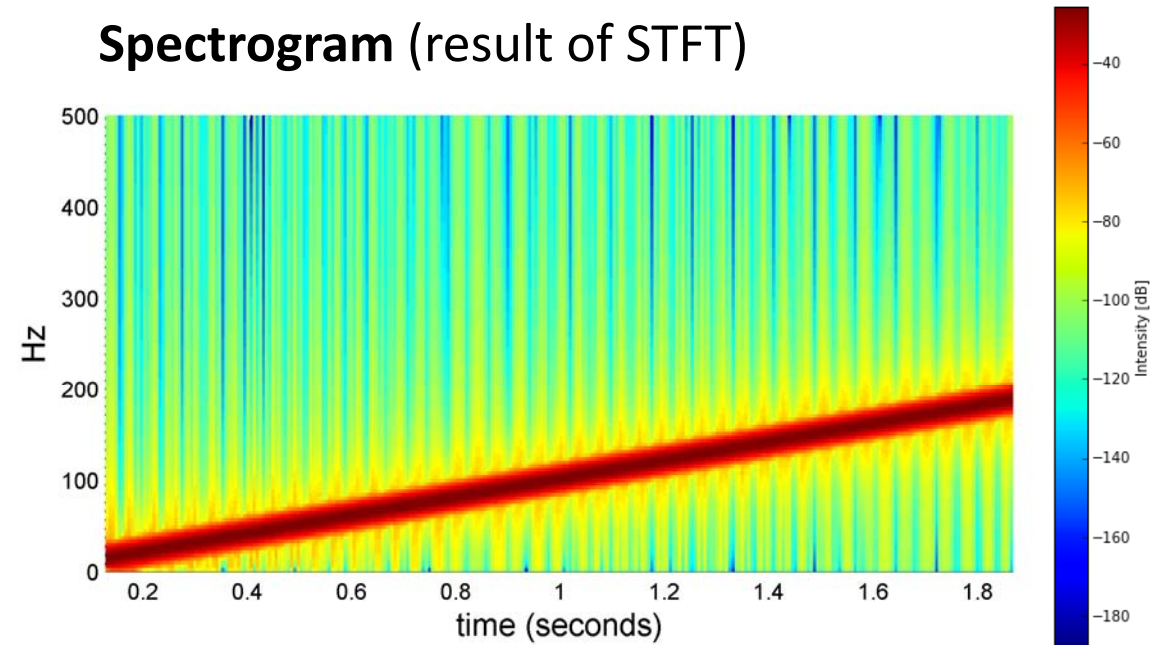


- **Waveform:** amplitude vs time
- **Spectrum:** amplitude vs frequency
- **Spectrogram:** time vs frequency; use *color* for amplitude

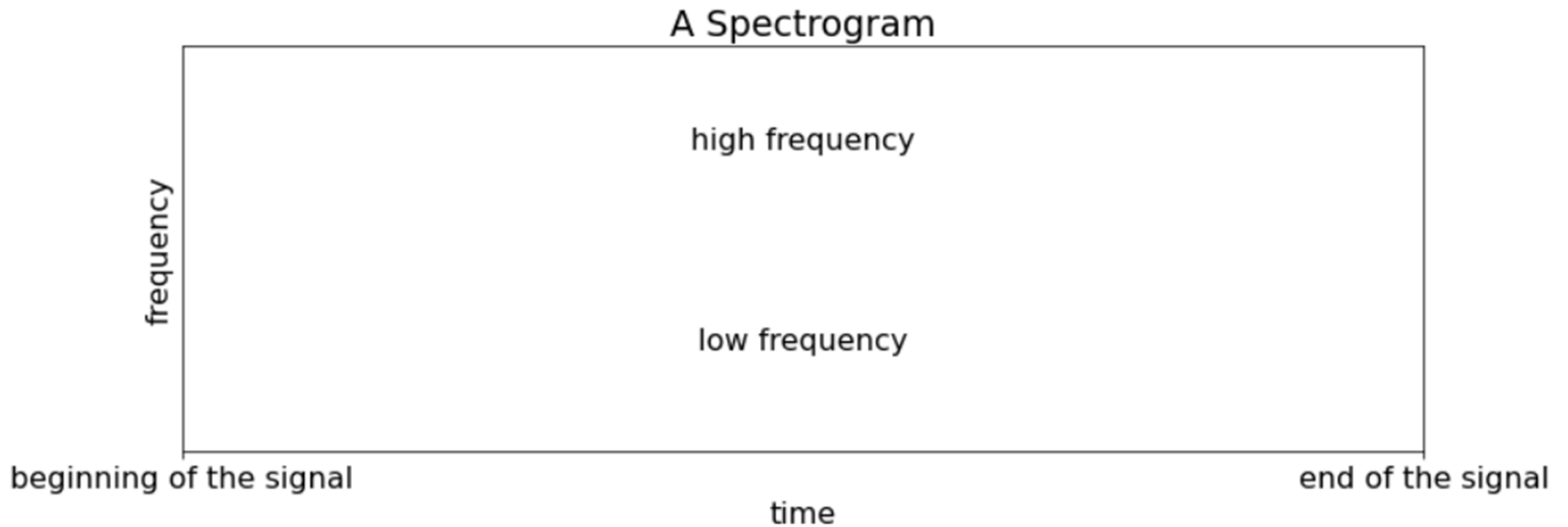
Spectrum (result of FFT)



Spectrogram (result of STFT)



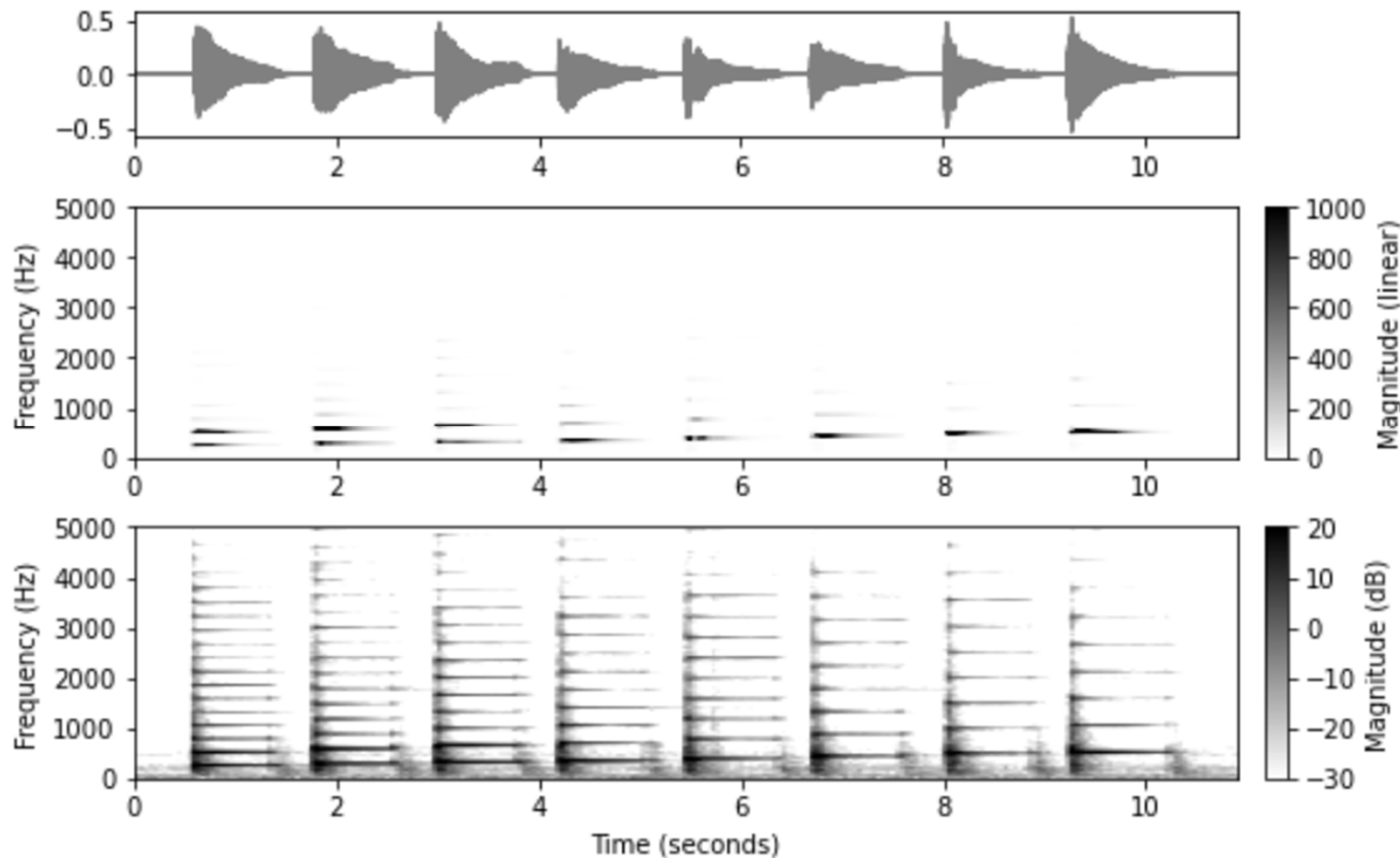
Time-Frequency Analysis via STFT



https://music-classification.github.io/tutorial/part2_basics/input-representations.html

Time-Frequency Analysis via STFT

https://www.audiolabs-erlangen.de/resources/MIR/FMP/C2/C2_STFT-Basic.html



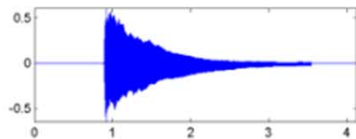
- **Time-frequency representation of audio**

- Linear frequency, **linear** magnitude
- Linear frequency, **logarithmic** magnitude (dB)

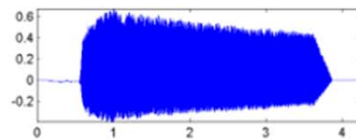
Timbre: Spectral Aspect

- Fundamental frequency (**F0**)
- **Partials** (harmonics): usually at frequencies which are **integer multiples** of F0
 - The partials in different instruments may exhibit **relative strengths**
 - The frequency may *deviate* from the ideal harmonics (this is called **inharmonic**ity)

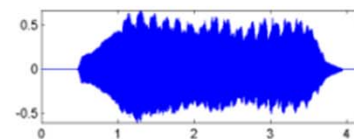
Piano



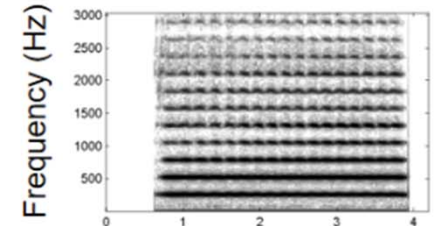
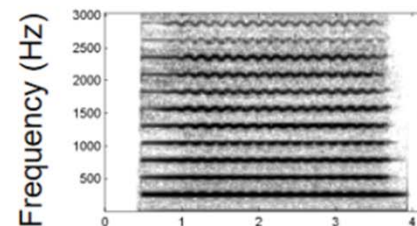
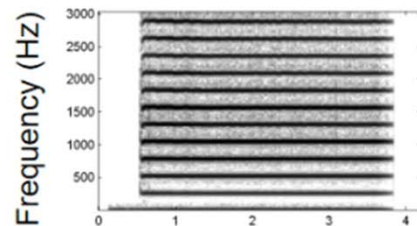
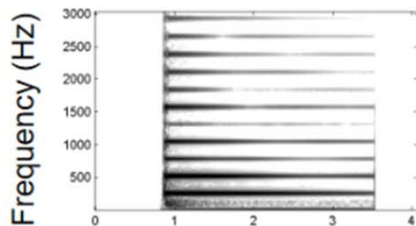
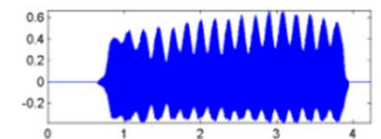
Trumpet



Violin



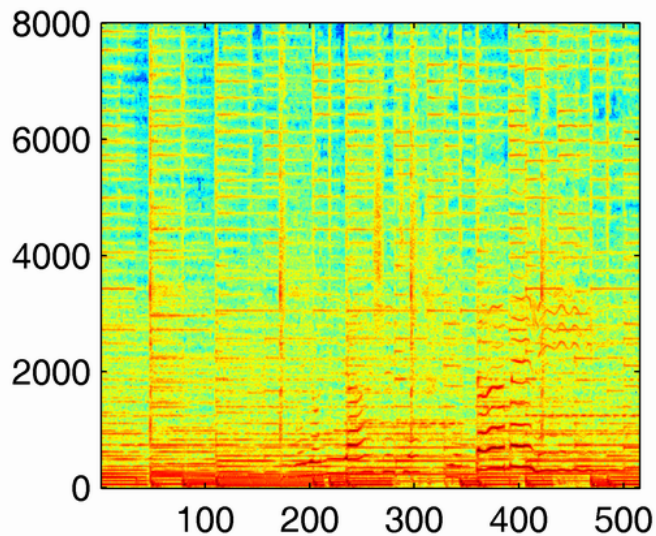
Flute



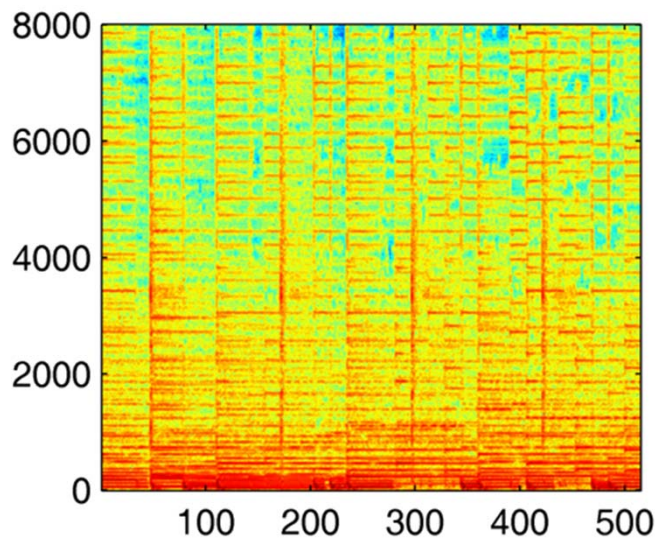
Time-Frequency Analysis via STFT

- For tasks such as classification/detection and source separation

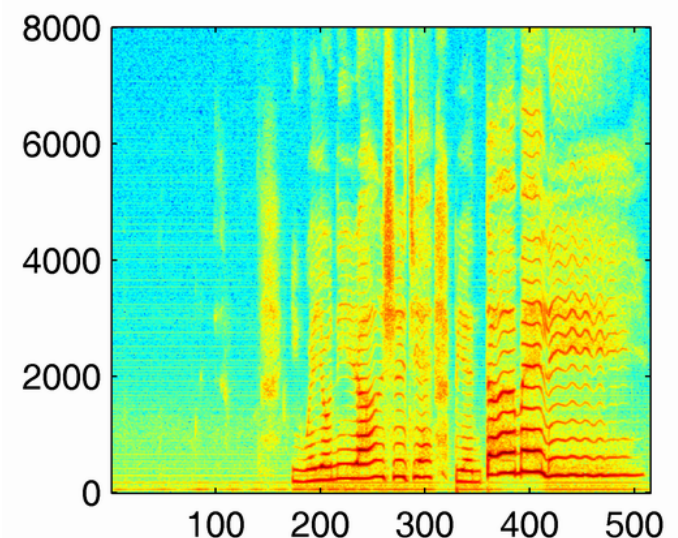
mixture



instrumental (clean)

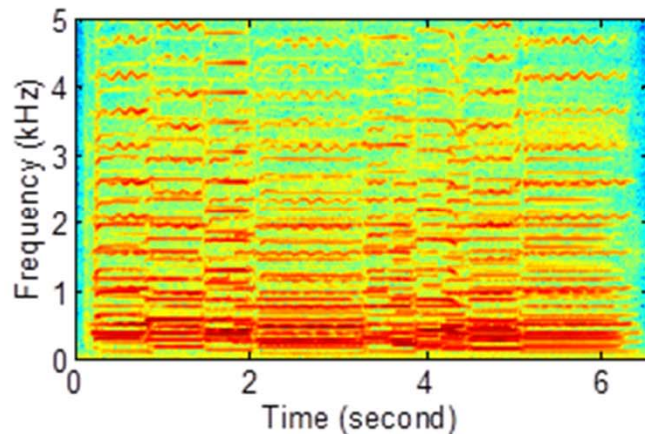


vocal (clean)

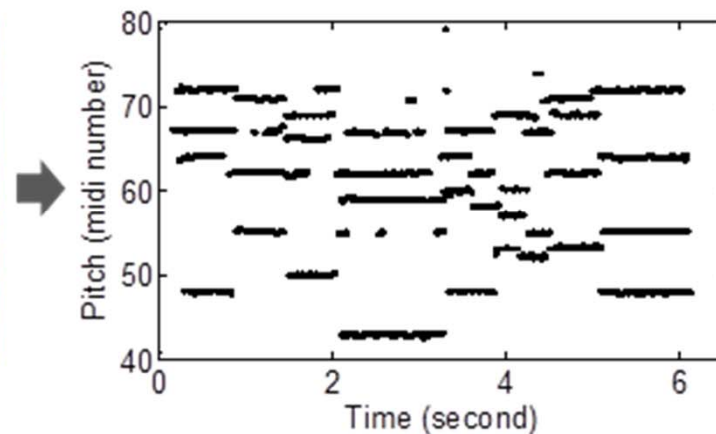


Time-Frequency Analysis via STFT

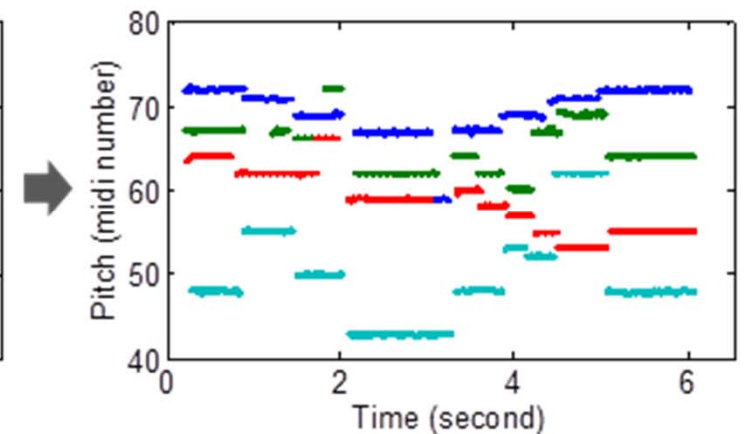
- For tasks such as multi-pitch estimation (MPE) and transcription



Spectrogram



Multi-pitch estimation results



MPE + instrument detection

<https://labsites.rochester.edu/air/projects/multipitch/multipitch.html>

torchaudio.transforms.Spectrogram

<https://docs.pytorch.org/audio/main/generated/torchaudio.transforms.Spectrogram.html>

```
waveform, sample_rate = torchaudio.load("test.wav", normalize=True)
transform = torchaudio.transforms.Spectrogram(n_fft=800)
spectrogram = transform(waveform)
```

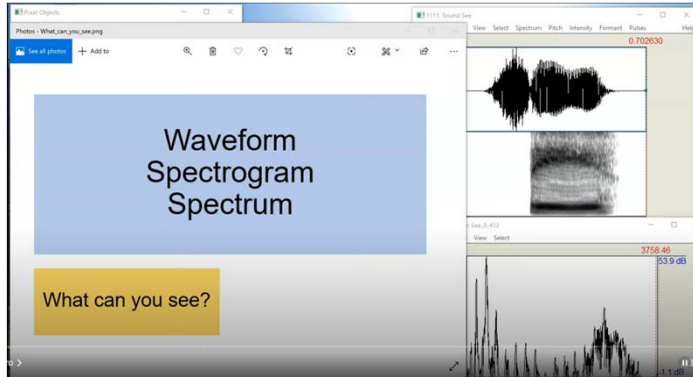
Parameters

- **n_fft** (*int*, optional) – Size of FFT, creates `n_fft // 2 + 1` bins. (Default: 400)
- **win_length** (*int* or *None*, optional) – Window size. (Default: `n_fft`)
- **hop_length** (*int* or *None*, optional) – Length of hop between STFT windows. (Default: `win_length // 2`)

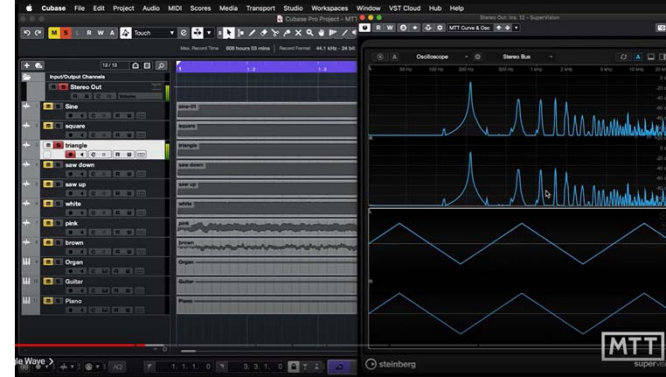
Source of Copyleft Audio Recordings

- <https://www.freesound.org/>
 - <https://www.freesound.org/people/acclivity/sounds/22347/>
 - https://www.freesound.org/people/Rudmer_Rotteveel/sounds/316915/
 - <https://www.freesound.org/people/Jaylew1987/sounds/321112/>
 - <https://www.freesound.org/people/mickel11/sounds/90803/>

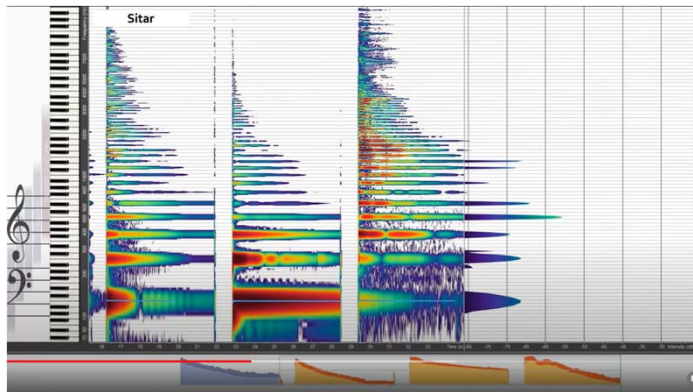
Related Videos



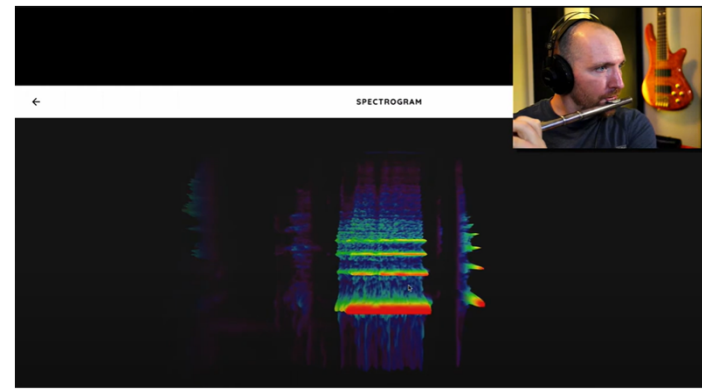
<https://www.youtube.com/watch?v=2Hj1kAWVjLo>



<https://www.youtube.com/watch?v=fZ18C5RXDcc>



<https://www.youtube.com/watch?v=VRAXK4QKJ1Q>

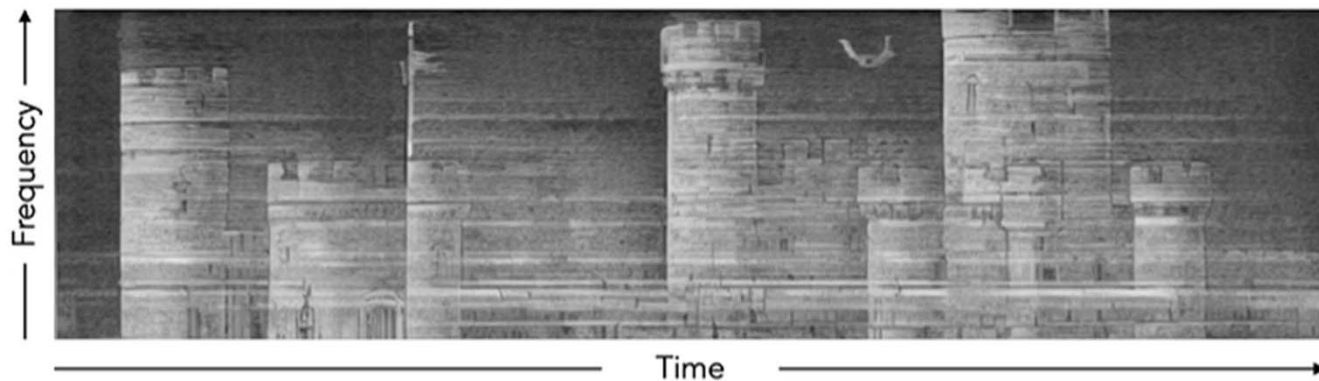


<https://www.youtube.com/watch?v=jEO0GHU3xsU>

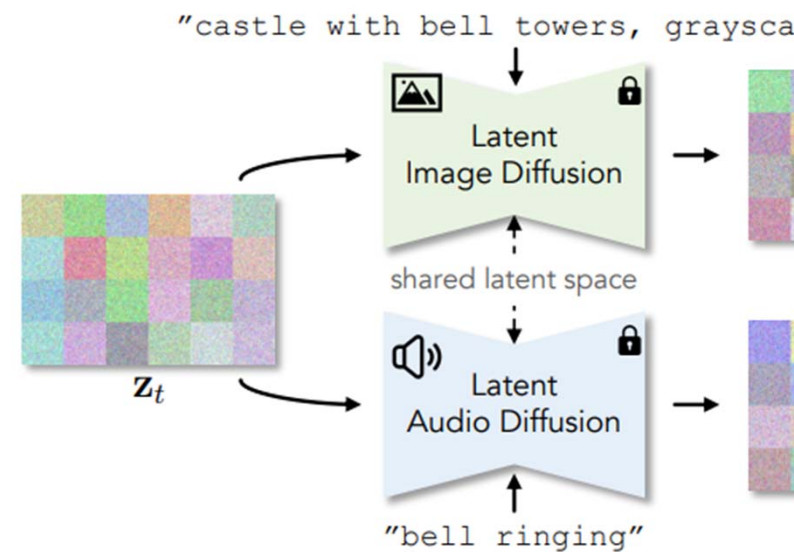
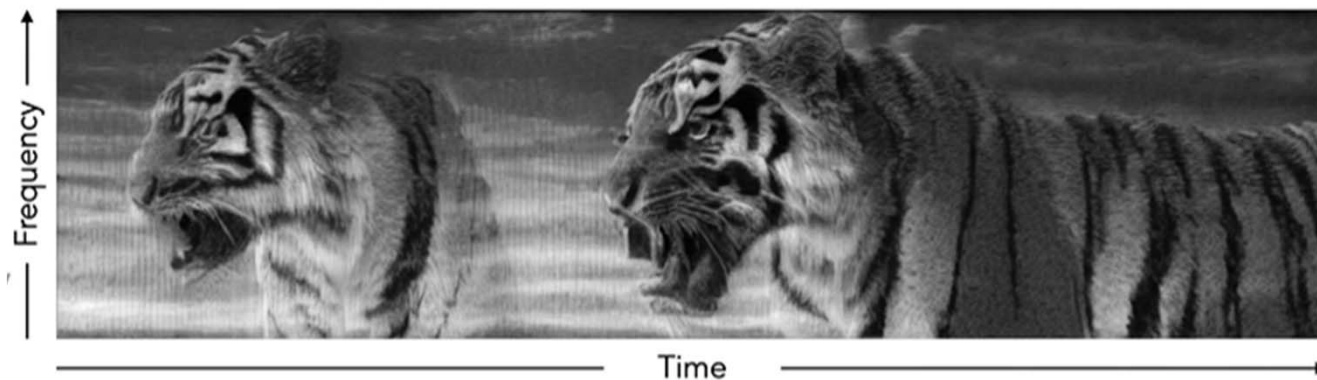
Fun Stuff: “Images that Sound”

<https://ificl.github.io/images-that-sound/>

🖼️ “castle with bell towers, grayscale, lithograph style”



🖼️ “tiger, grayscale, black background”



What Do We Perceive in Musical Audio?

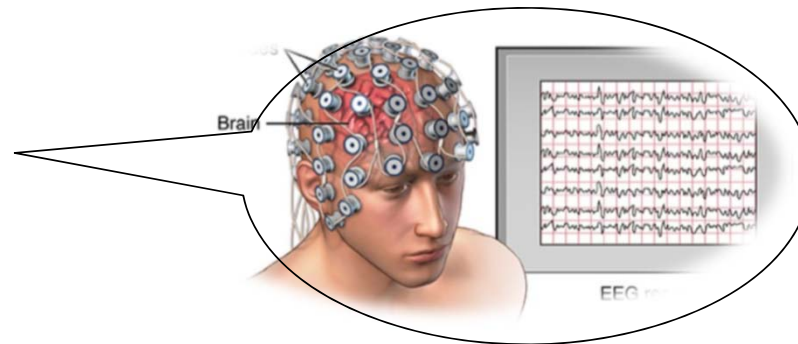
- Loudness
- Spatial information
- Timbre / instrument / singer identity
- Pitch / melody
- Chord / harmony
- Tempo / rhythm
- Style
- Emotion
- Others (e.g., “The power of music compounds over time through repeated listening. Culture builds around shared experiences.”)

Outline

- Time and frequency representations for music
- **Math in STFT**

Sampling Rate

- Definition: number of samples per second
- Why: analog to digital
- Examples
 - EEG signal: **128** Hz
 - Telephone audio: **8k** Hz
 - Music audio: **44k** HZ

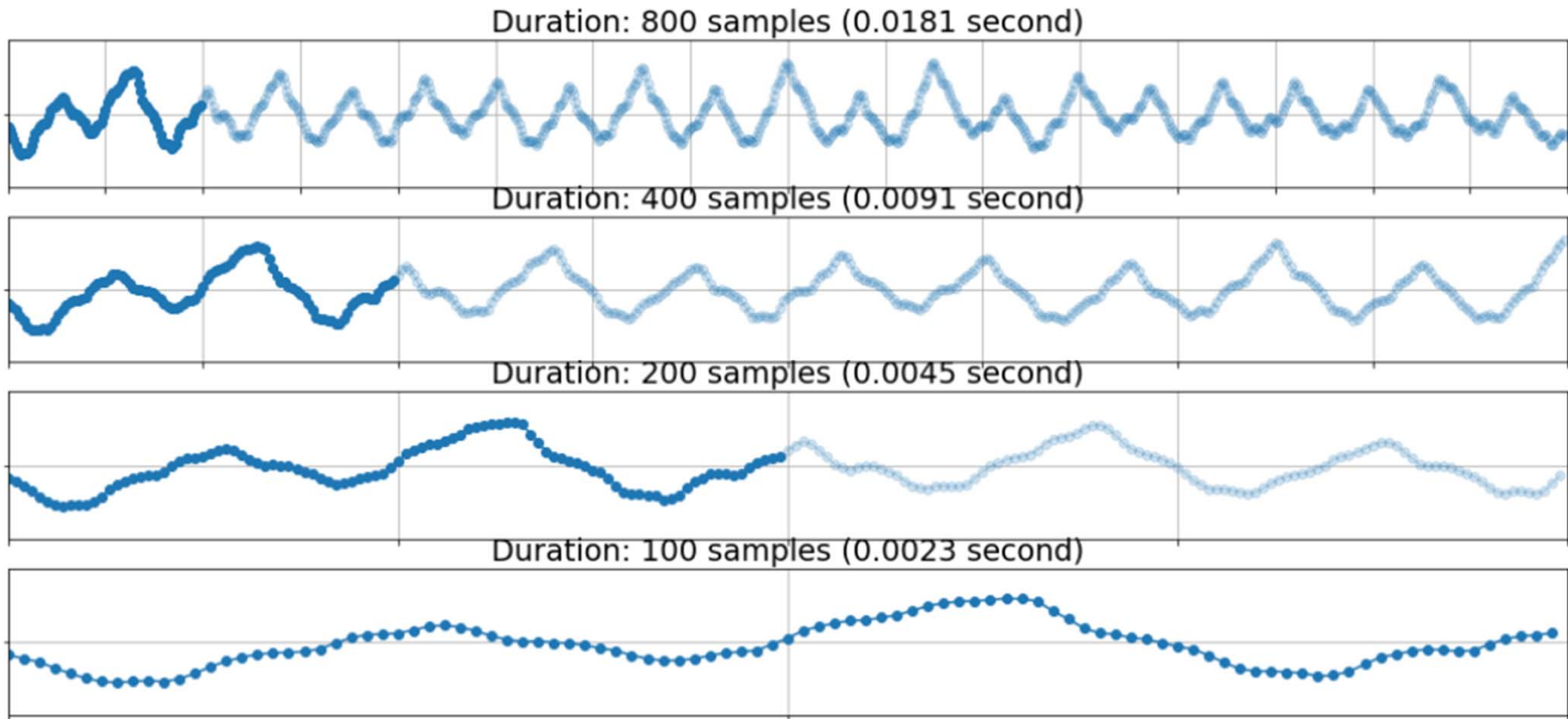


<https://www.brightbraincentre.co.uk/electroencephalogram-eeeg-brainwaves/>

```
import soundfile as sf
import pyloudnorm as pyln

data, rate = sf.read("test.wav") # load audio
```

Sampling Rate



https://music-classification.github.io/tutorial/part2_basics/input-representations.html

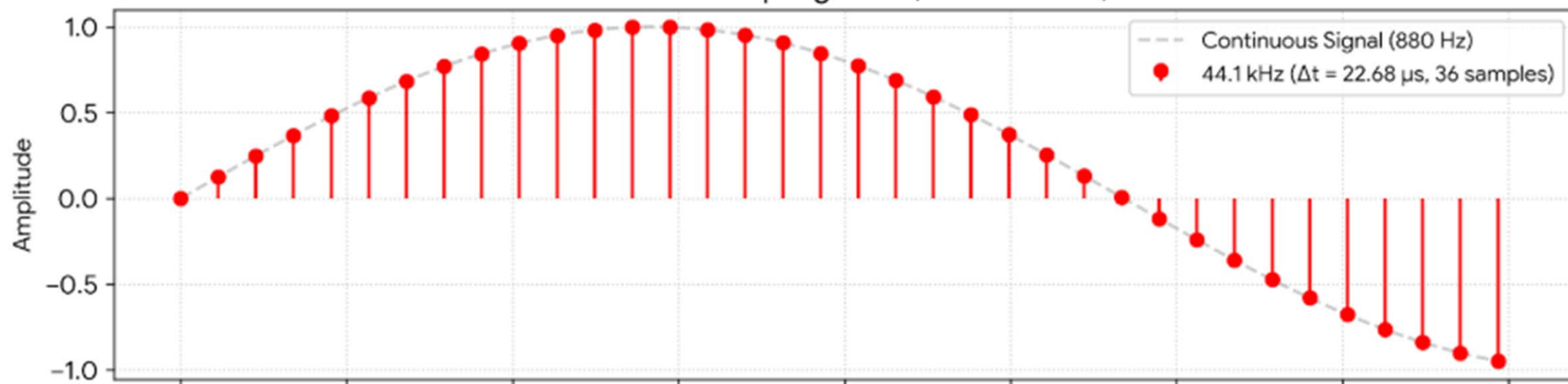
Sampling Rate

- Usually, a model considered signals with a **certain and fixed** sampling rate
 - Data with different sampling rates are not on the same time basis

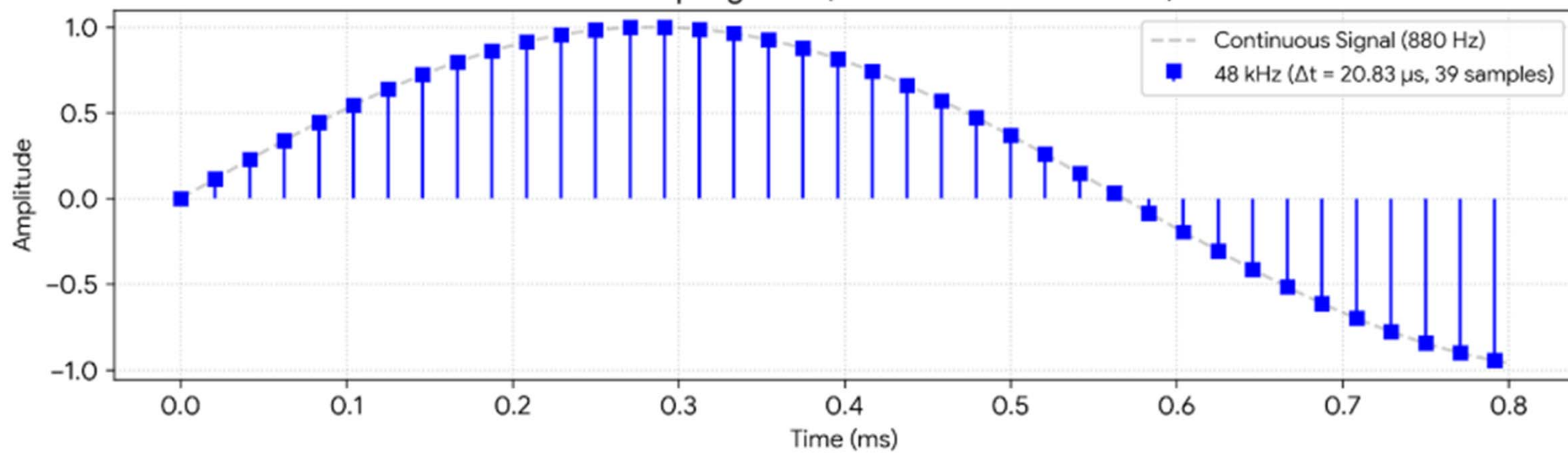
```
data, rate = sf.read("test.wav") # load audio
```

- Make sure your data are under **the same** sampling rate
- Double check the sampling rate assumed by the models you sourced from GitHub
 - Speech: 16k Hz
 - Music: 22k, 44k, or 48k Hz
- If we want to ML/DL model trained on signals with one sampling rate to process a signal under a different sampling rate, **“resampling”** of that signal is needed
 - Upsampling
 - Anti-aliasing (low-pass filtering)
 - Downsampling

44.1 kHz Sampling Rate (CD Standard)



48 kHz Sampling Rate (Studio / Video Standard)



Resampling

https://docs.pytorch.org/audio/stable/tutorials/audio_resampling_tutorial.html

```
resampler = torchaudio.transforms.Resample( orig_freq=orig_freq,  
new_freq=new_freq )
```

```
resampled_waveform = resampler(waveform)
```

Library	Time on CPU (ms)
soxr (HQ)	7.2
scipy.signal.resample	13.4
soxr (VHQ)	15.8
torchaudio	19.2

Sample Rate independent Models

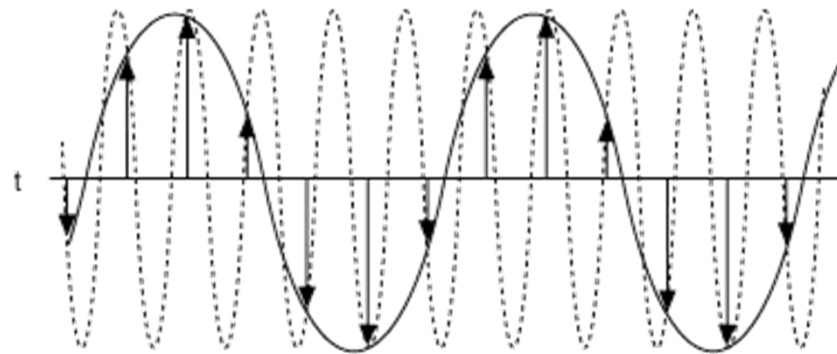
- Usually, a model considered signals with a certain and fixed sampling rate
- Building a **sampling-rate agnostic** model that can take signals sampled at arbitrary sampling rates is an advanced topic

Ref 1: Saito et al, "Sampling-frequency-independent audio source separation using convolution layer based on impulse invariant method," EUSIPCO 2021

Ref 2: Carson et al, "Sample rate independent recurrent neural networks for audio effects processing," DAFx 2024

Nyquist–Shannon Sampling Theorem

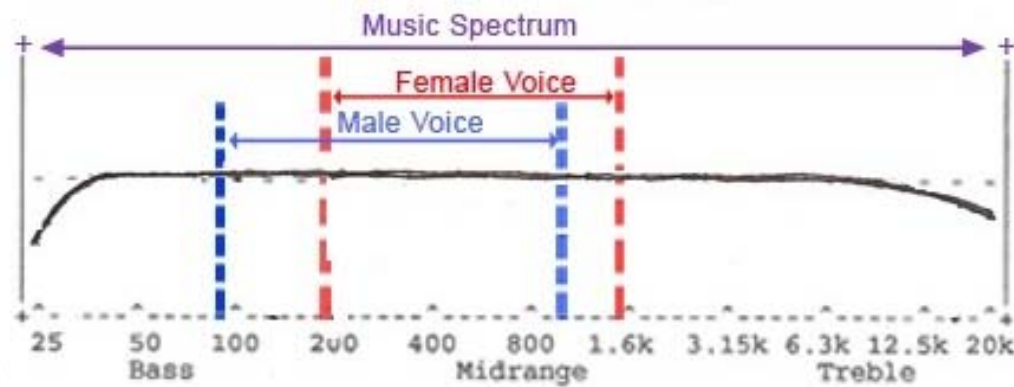
- A signal must be sampled at least **twice** as fast as the bandwidth of the signal to accurately reconstruct the waveform; otherwise, the high-frequency content will alias at a frequency inside the spectrum of interest
- **Sampling freq (F_s) > 2* the highest freq in the signal**



http://zone.ni.com/reference/en-XX/help/370524T-01/siggenhelp/fund_nyquist_and_shannon_theorems/

Nyquist–Shannon Sampling Theorem

- Telephone audio: 8k Hz
 - Via phone, we cannot hear frequency higher than 4k Hz



<https://www.quora.com/How-do-HRT-sex-reassignment-and-other-such-procedures-affect-vocal-production-particularly-the-singing-voice>

- **Question:** With $F_s=128$ Hz, we assume that we don't need to care about frequency higher than ___ Hz in brain waves

Nyquist–Shannon Sampling Theorem

Brainwaves, Frequencies and Functions

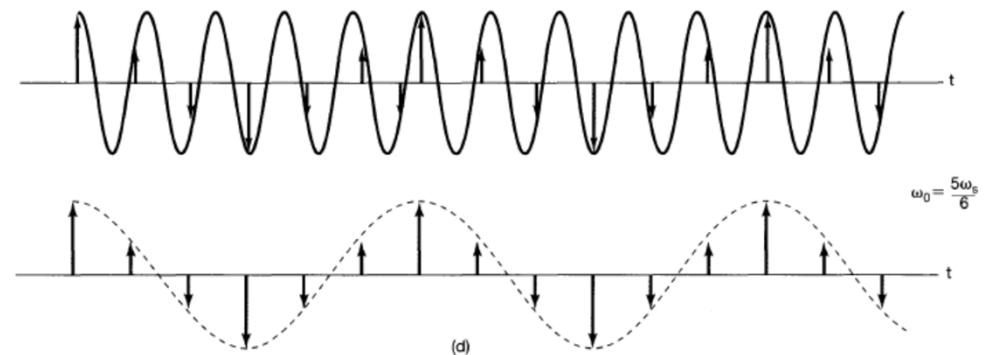
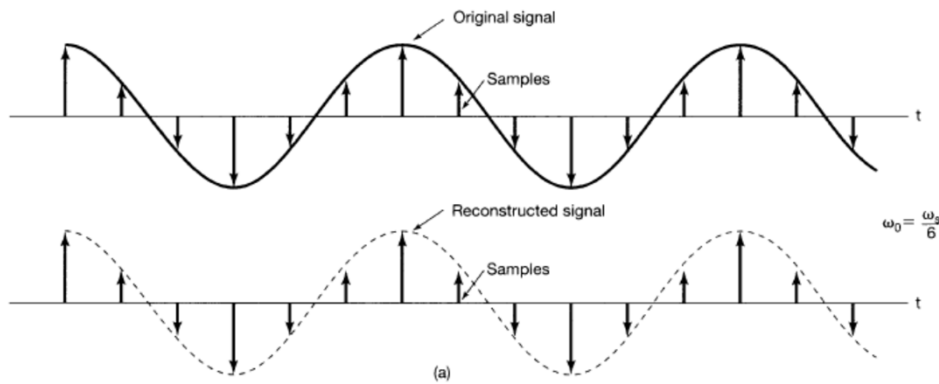
Unconscious		Conscious		
Delta	Theta	Alpha	Beta	Gamma
0,5 – 4 Hz	4 – 8 Hz	8 – 13 Hz	13 – 30 Hz	30-42 Hz
Instinct	Emotion	Consciousness	Thought	Will
Survival Deep sleep Coma	Drives Feelings Trance Dreams	Awareness of the body Integration of feelings	Perception Concentration Mental activity	Extreme focus Energy Ecstasy

<http://altered-states.net/barry/update236/>

- **Question:** With $F_s=128$ Hz, we assume that we don't need to care about frequency higher than 64 Hz in brain waves

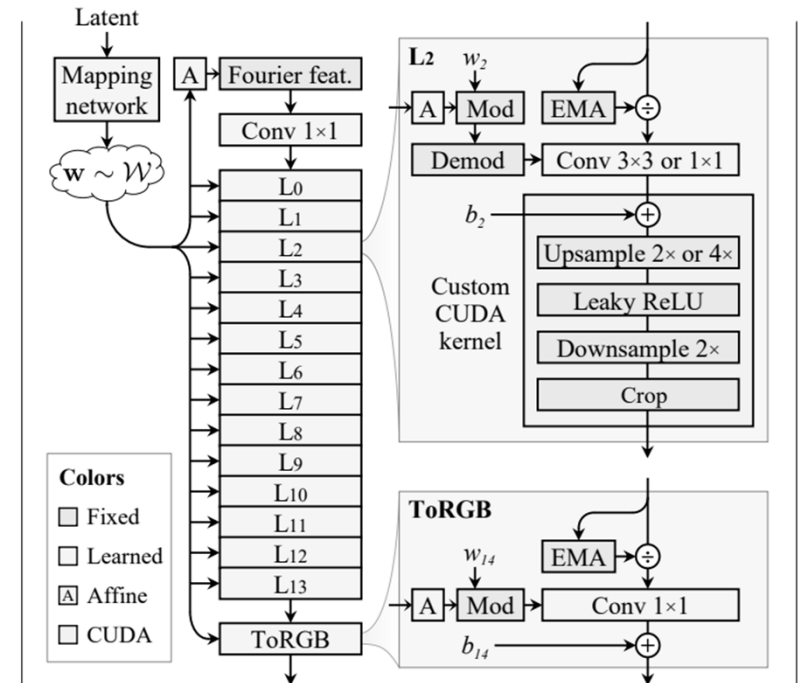
Aliasing

- *High-frequency* information has already **disguised itself** as (“aliased into”) a *low-frequency* signal at discrete sample points
- The values of high-frequency and low-frequency components become completely identical at discrete sampling points



Aliasing

- A neural network may perform lots of **downsampling** (e.g., pooling)
- Special care is needed for anti-aliasing, e.g., via low-pass filtering (**LPF**)
 - So that high-frequency information is “**removed**” rather than aliased into the low-frequency part



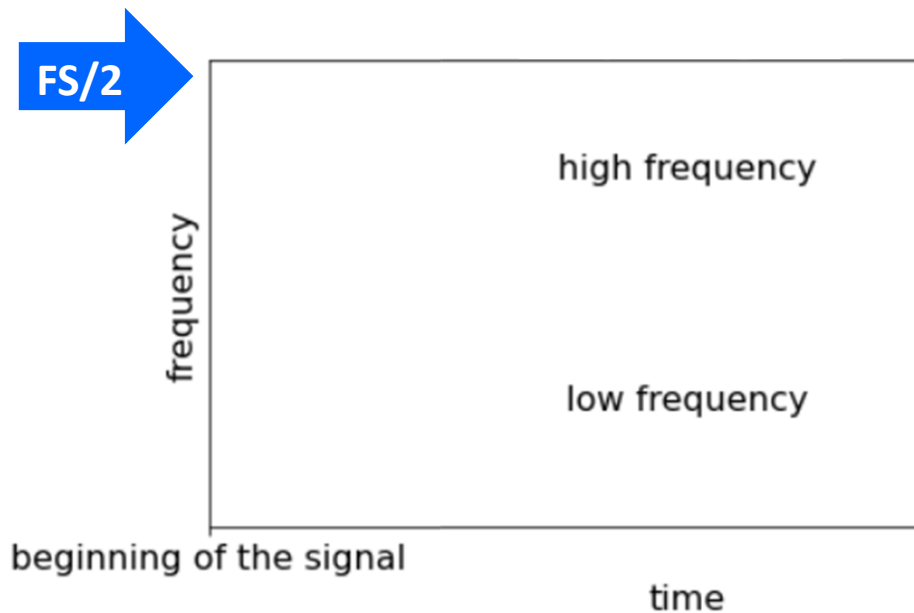
(b) Our alias-free StyleGAN3 generator architecture

<https://nvlabs.github.io/stylegan3/>

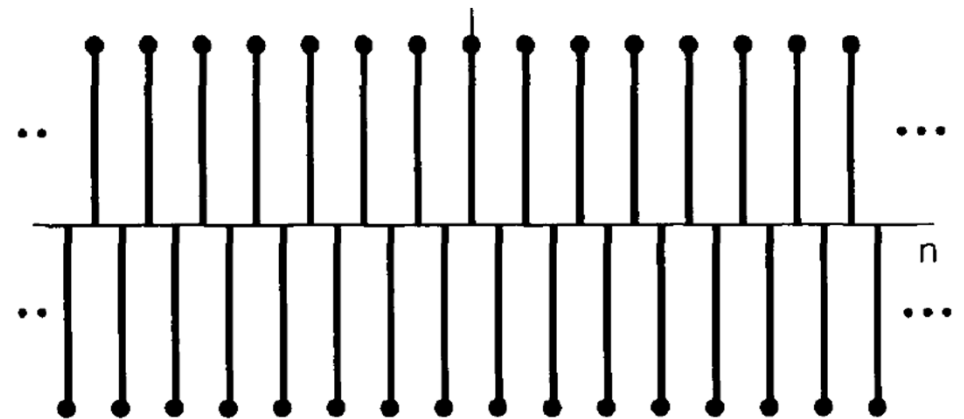
Ref 2: Karras et al, "Alias-free generative adversarial networks," NeurIPS 2021

Nyquist–Shannon Sampling Theorem

- Sampling freq (F_s) $> 2 \times$ the highest freq in the signal
- **Highest freq bin in STFT = $F_s/2$**



Sinusoid with the highest freq given a fixed F_s



DTFS: Fourier Series Representation for Discrete-time Periodic Signals

- For **periodic** signals with fundamental **period N** , i.e., $x[n] = x[n + N]$
- The Fourier series coefficients a_k can be interpreted as the **inner product** of the signal $x[n]$ and a basis function $e^{jk\frac{2\pi}{N}n}$, which is complex-valued
- a_k are **complex-valued**
- a_k are also **periodic with period N** , i.e., $a_k = a_{k+N}$

Synthesis equation: $x[n] = \sum_{k=\langle N \rangle} a_k e^{jk\frac{2\pi}{N}n}$

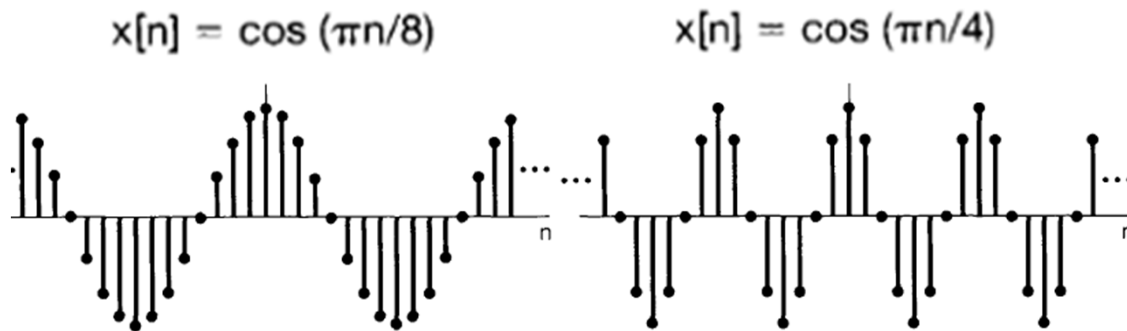
Analysis equation: $a_k = \frac{1}{N} \sum_{n=\langle N \rangle} x[n] e^{-jk\frac{2\pi}{N}n}$

In discrete-time, $e^{jk\omega n} = e^{j(k+N)\omega n}$; so there are **only N unique basis vectors**

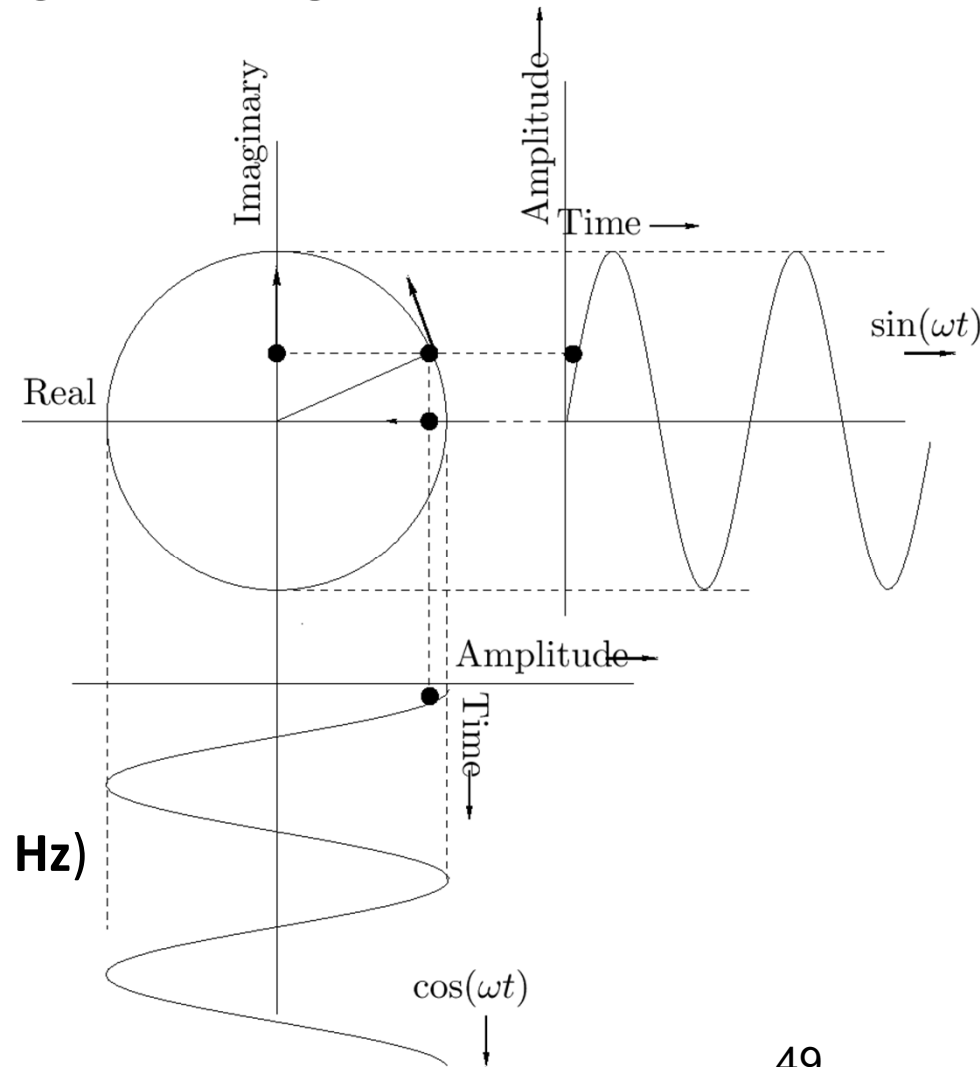
Sum over a set of N successive integers

The Basis Functions in DTFS

- $e^{j\omega n} = \cos(\omega n) + j\sin(\omega n)$



- angular or ordinary frequency
 - ω : angular frequency (radians per second)
 - f : ordinary frequency (cycles per second; in Hz)
 - $e^{j2\pi f n} = \cos(2\pi f n) + j\sin(2\pi f n)$



Discrete-time Fourier Transform (DFT): Extending DTFS to Deal with Aperiodic, Finite-Duration DT Signals

- For **periodic DT** signal $x[n] \xleftrightarrow{\mathcal{FS}} a_k$

Synthesis: $x[n] = \sum_{k=\langle N \rangle} a_k e^{jk\omega n}$

Analysis: $a_k = \frac{1}{N} \sum_{n=\langle N \rangle} x[n] e^{-jk\omega n}$

- For **aperiodic, finite-duration DT** signal of length L , $x[n] \xleftrightarrow{\mathcal{DFT}} X_k$

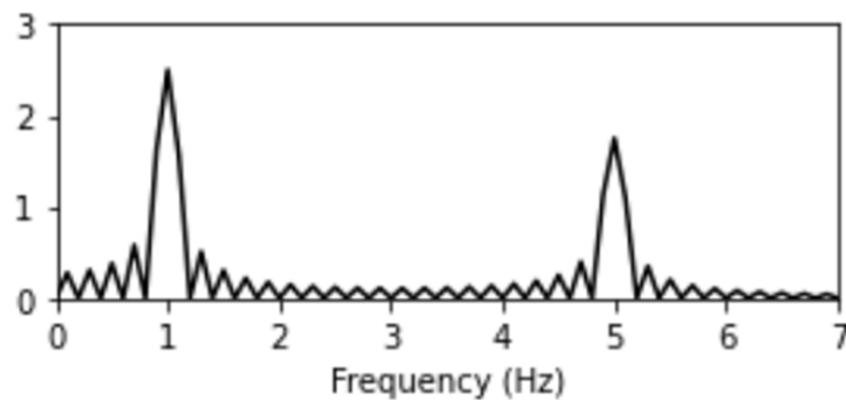
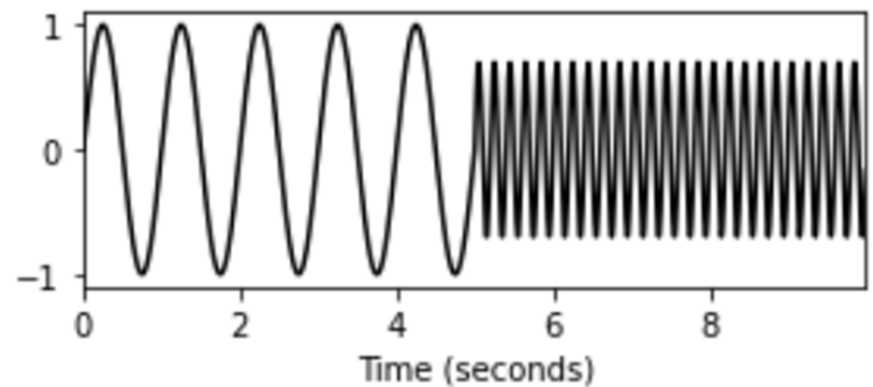
Synthesis: $x[n] = \sum_{k=0}^{L-1} X_k e^{jk\omega n}$

Analysis: $X_k = \frac{1}{L} \sum_{n=0}^{L-1} x[n] e^{-jk\omega n}$

NOTE: While the length of $x[n]$ is L , the length of X_k is also L

However, FFT cannot Localize Temporal Events

https://www.audiolabs-erlangen.de/resources/MIR/FMP/C2/C2_STFT-Basic.html



STFT: Short-time Fourier Transform

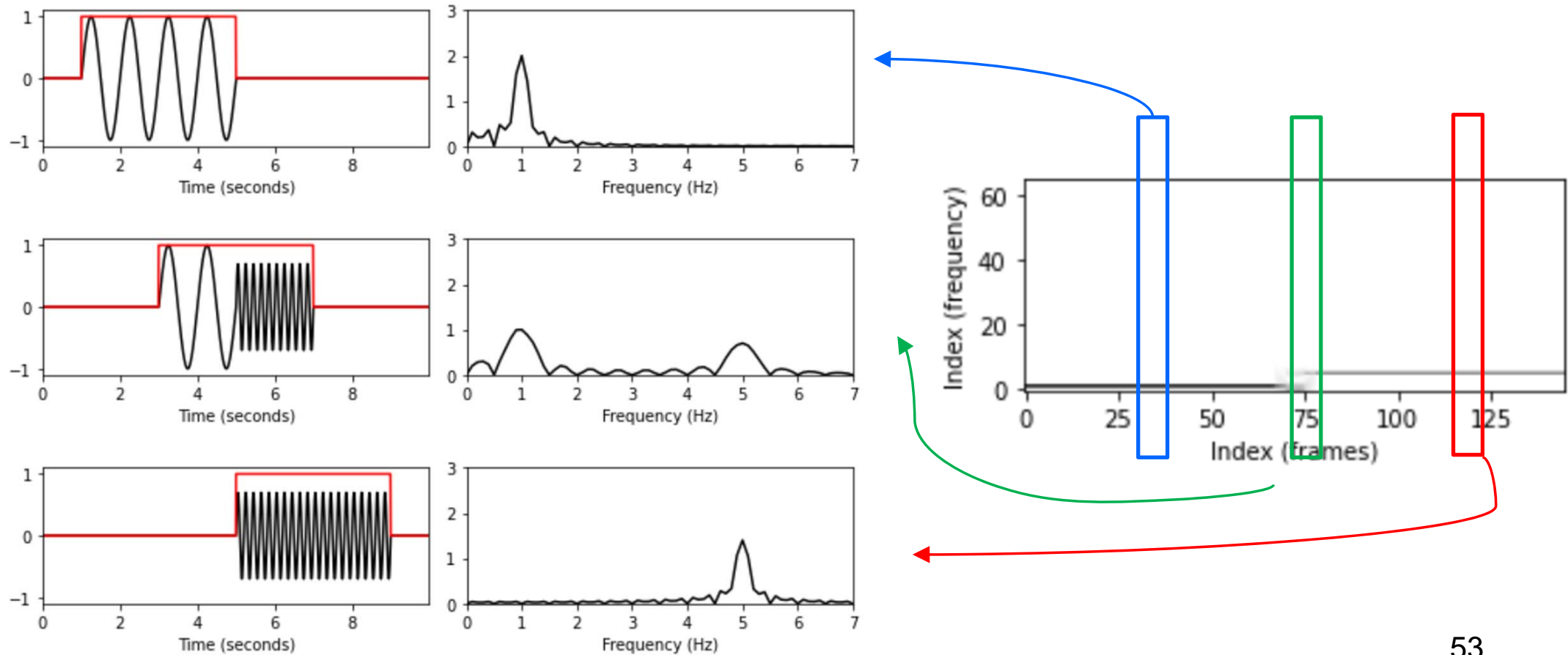
https://www.audiolabs-erlangen.de/resources/MIR/FMP/C2/C2_STFT-Basic.html

$$\mathcal{X}(m, k) := \sum_{n=0}^{N-1} x(n + mH)w(n)\exp(-2\pi i kn/N)$$

- $x[n]$: real-valued discrete-time signal of length L obtained by equidistant sampling with respect to a fixed sampling rate
- $w[n]$: window function of length $N_{\text{win}} < L$
- H : “hop size”
- **STFT**: a sequence of **localized DFTs**

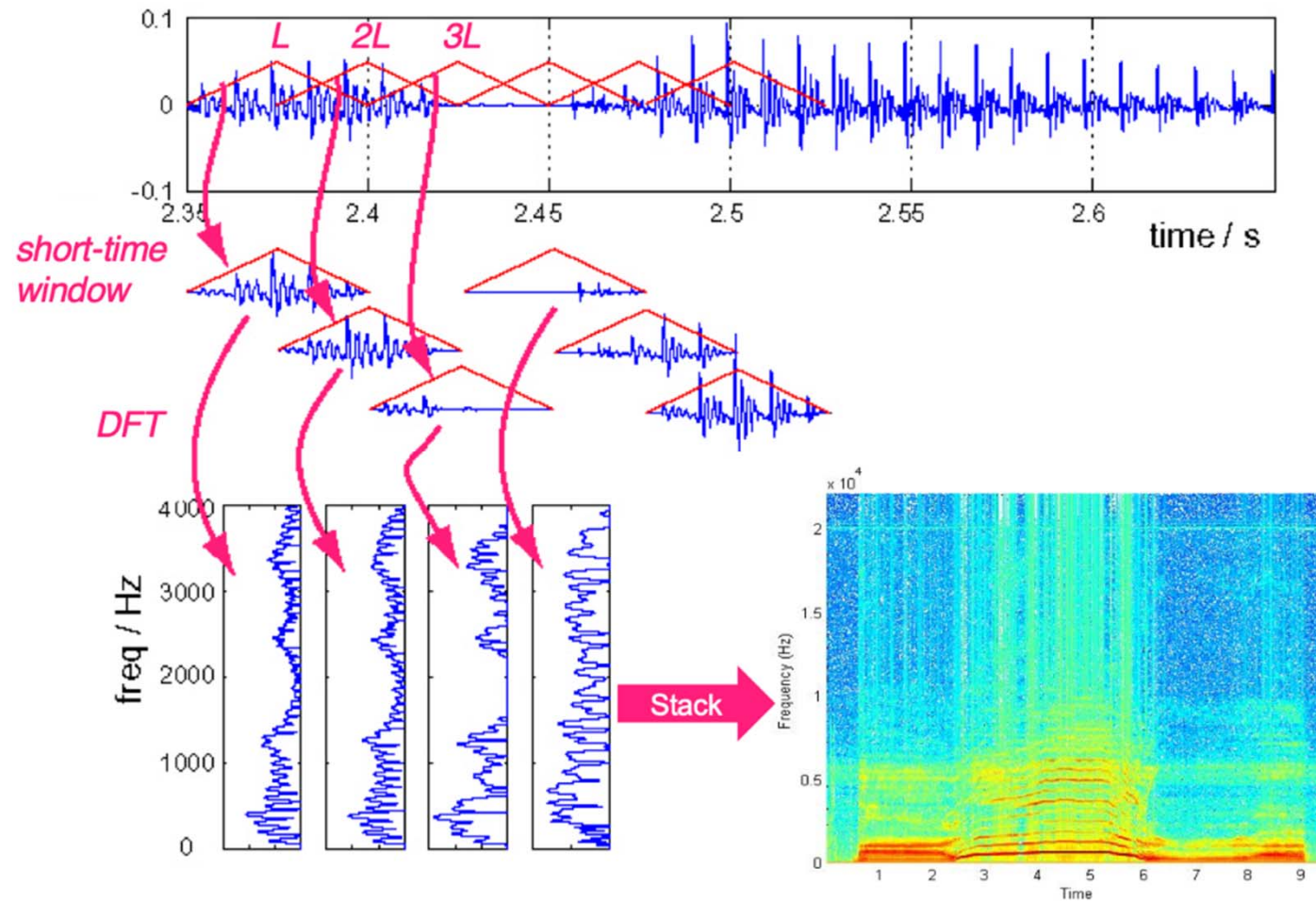
Windowed FFT

https://www.audiolabs-erlangen.de/resources/MIR/FMP/C2/C2_STFT-Basic.html



Windowed FFT

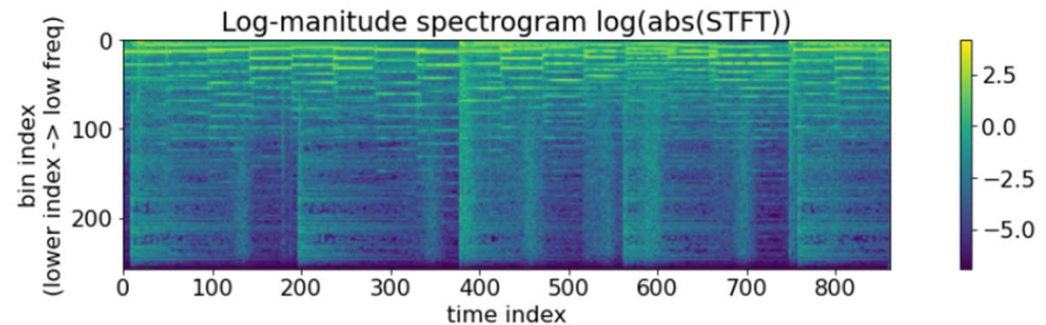
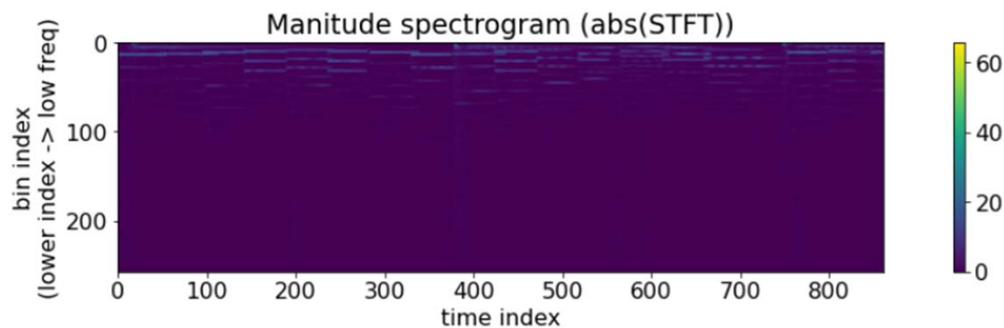
<https://source-separation.github.io/tutorial/basics/representations.html>



Magnitude Spectrogram

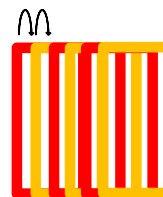
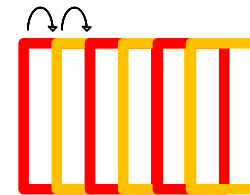
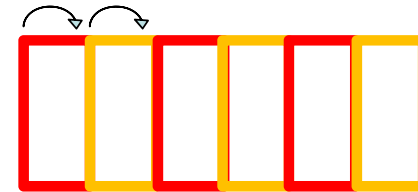
https://music-classification.github.io/tutorial/part2_basics/input-representations.html?highlight=phase

- **STFT**: complex-valued, linear frequency scale
- **Spectrogram**: a time-frequency representation, usually computed via STFT
- **Magnitude spectrogram**: $\text{abs}(\text{STFT})$; real-valued, non-negative
- **Log-magnitude spectrogram**: $\log(\text{abs}(\text{STFT}))$; real-valued, non-negative
- The **phase** information of STFT is often considered less important



Hop Size in STFT

- Hop size
 - Can be proportional to the window size
 - $\text{hop_size} = \text{win_size}$
 - **$\text{hop_size} = 0.5 * \text{win_size}$**
 - $\text{hop_size} = 0.1 * \text{win_size}$
 - Can also be not proportional to the window size



torchaudio.transforms.Spectrogram

<https://docs.pytorch.org/audio/main/generated/torchaudio.transforms.Spectrogram.html>

```
waveform, sample_rate = torchaudio.load("test.wav", normalize=True)
transform = torchaudio.transforms.Spectrogram(n_fft=800)
spectrogram = transform(waveform)
```

Parameters

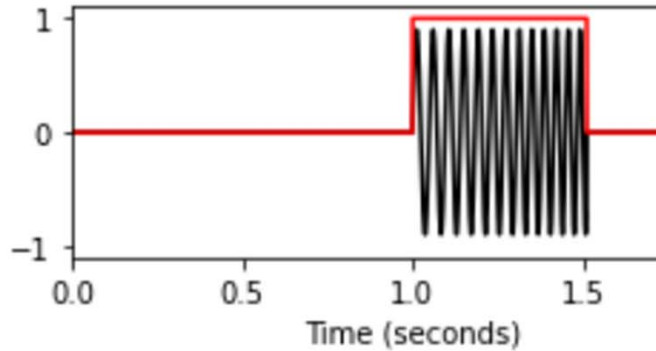
- **n_fft** (*int*, optional) – Size of FFT, creates $n_fft // 2 + 1$ bins. (Default: 400)
- **win_length** (*int* or *None*, optional) – Window size. (Default: `n_fft`)
- **hop_length** (*int* or *None*, optional) – Length of hop between STFT windows. (Default: `win_length // 2`)

Window Functions in STFT

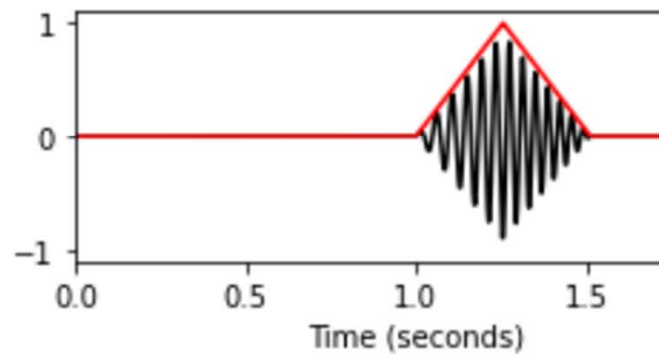
https://www.audiolabs-erlangen.de/resources/MIR/FMP/C2/C2_STFT-Window.html

- Hann window is often used

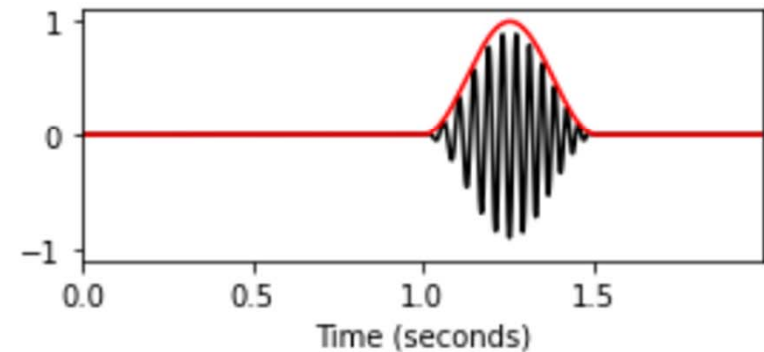
Rectangular window:



Triangular window:

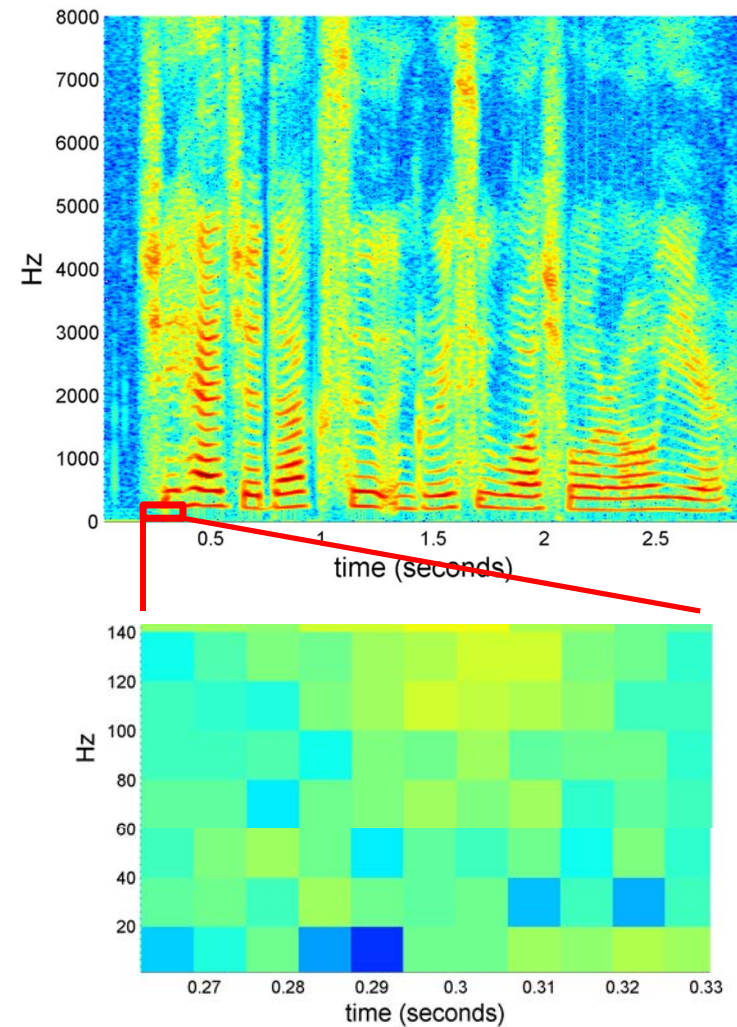


Hann window:



Frame, Frequency Bin, and Time-Frequency Point

- The magnitude spectrogram can be represented as a **matrix**
- **Frame**: along time
- **Frequency bin**: along frequency
- **Time-frequency point**

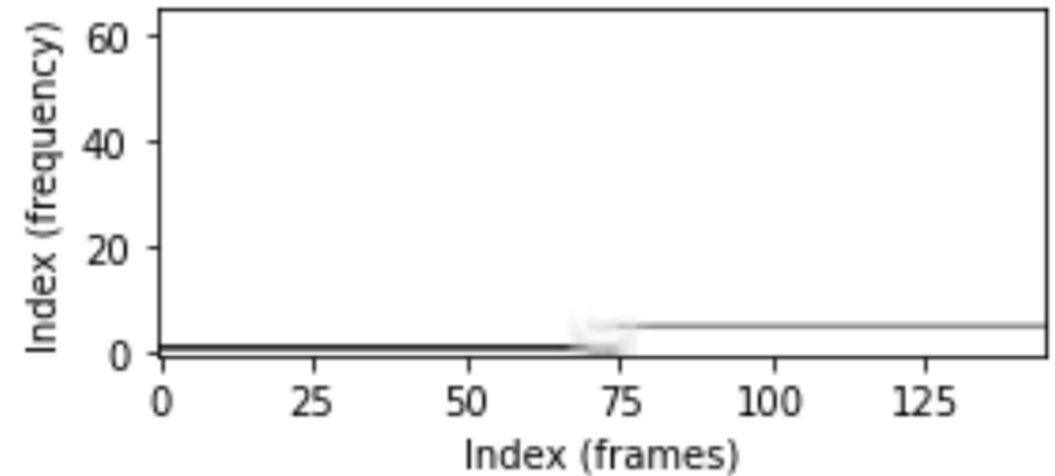


Interpretation of Time and Frequency Indices in STFT

https://www.audiolabs-erlangen.de/resources/MIR/FMP/C2/C2_STFT-Basic.html

$$F_{\text{coef}}(k) := \frac{k \cdot F_s}{N}$$

- **Frequency spacing** (Δfreq): $\frac{F_s}{N}$
- **Temporal spacing** (Δtime): $\frac{H}{F_s}$



$$T_{\text{coef}}(m) := \frac{m \cdot H}{F_s}$$

$$\text{Frequency spacing } (\Delta\text{freq}) = \frac{F_s}{N}$$

- $a_k = \frac{1}{N} \sum_{n=\langle N \rangle} x[n] e^{-jk \frac{2\pi}{N} n}$
 - $a_k = a_{k+N}$ for every signal
 - $a_k = a_{-k}^* = a_{N-k}^*$ for real signals due to conjugate property (thus, $|a_k| = |a_{N-k}|$)
- For real signals, we only need to consider $k = 0, \dots, \frac{N}{2}$
 - $a_0 = \frac{1}{N} \sum_{n=\langle N \rangle} x[n]$
 - $a_1 = \frac{1}{N} \sum_n x[n] e^{-j \frac{2\pi}{N} n}$ (the period of the basis function is N ; lowest frequency)
 - $a_{\frac{N}{2}} = \frac{1}{N} \sum_n x[n] e^{-j\pi n} = \frac{1}{N} \sum_n x[n] (-1)^n$ (the period is 2; highest frequency)
 - In total, $\frac{N}{2} + 1$ points for positive frequency

Size of the Magnitude Spectrogram

- Number of frequency bins (i.e., number of rows in the matrix): $\left\lfloor \frac{N}{2} \right\rfloor + 1$
 - If N is even, this is $\frac{N}{2} + 1$
 - If N is odd, this is $\frac{N+1}{2}$
 - Only need to consider up to $\left\lfloor \frac{N}{2} \right\rfloor + 1$ because the magnitude of DFT output is symmetric for real-valued input signals
- Number of time frames (columns in the matrix): $\left\lfloor \frac{L-N}{H} \right\rfloor + 1$
 - L is the total number of samples in the signal
 - The $+1$ accounts for the first window starting at the beginning of the signal (i.e., initial frame at offset 0)

torchaudio.transforms.Spectrogram

<https://docs.pytorch.org/audio/main/generated/torchaudio.transforms.Spectrogram.html>

```
waveform, sample_rate = torchaudio.load("test.wav", normalize=True)
transform = torchaudio.transforms.Spectrogram(n_fft=800)
spectrogram = transform(waveform)
```

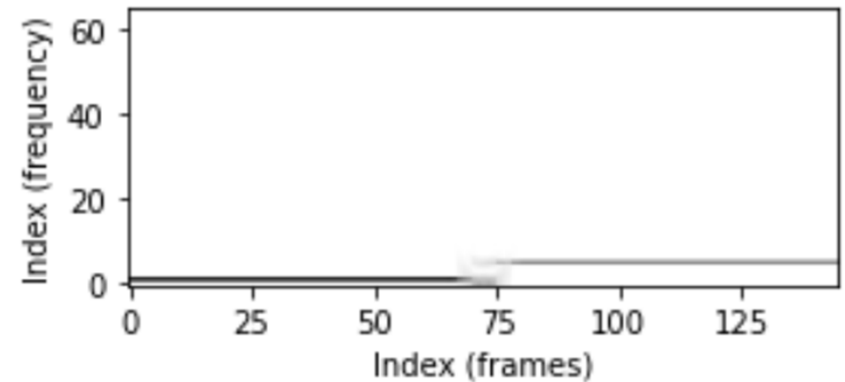
Parameters

- **n_fft** (*int, optional*) – Size of FFT, creates $n_fft // 2 + 1$ bins. (Default: 400)
- **win_length** (*int or None, optional*) – Window size. (Default: n_fft)
- **hop_length** (*int or None, optional*) – Length of hop between STFT windows. (Default: win_length // 2)

How about Frequencies in Between?

$$F_{\text{coef}}(k) := \frac{k \cdot F_s}{N}$$

- First frequency bin ($k = 0$): DC
- Second bin ($k = 1$): $\frac{F_s}{N}$ Hz
- An **arbitrary frequency** f rarely falls perfectly on a DFT grid point ($f \neq k \frac{F_s}{N}$)
 - DFT measures how much the input correlates with exact integer harmonic sinusoids
 - the energy for the component at f **spreads across all DFT bins** (“spectral leakage” due to the time-domain windowing operation [“frequency-domain conv with a sinc”])
 - the bin **closest** to the true frequency, $k = \text{round}\left(\frac{f \cdot N}{F_s}\right)$, contains the maximum energy peak, though it does not capture the full signal alone

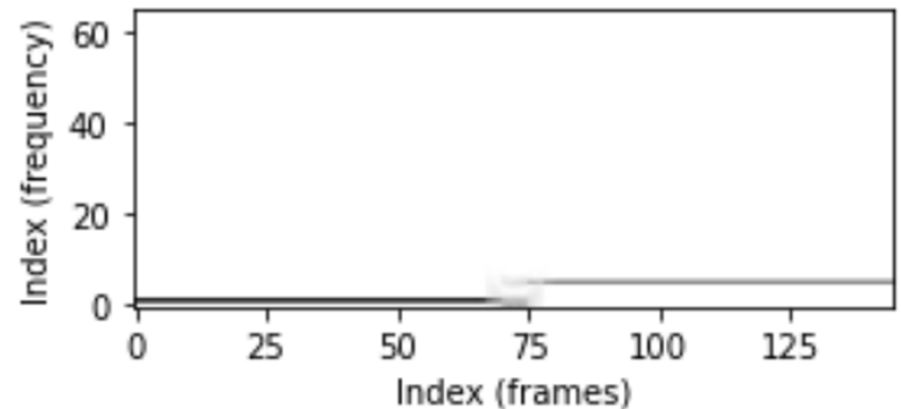


How about Frequencies in Between?

- Example

If $F_s = 44100$ Hz and $N = 1024$

- First bin: 0 Hz
- Second bin: 43.066 Hz
- Third bin: 86.132 Hz
- 20Hz $\rightarrow$ peak bin is $k = 0$
- 30Hz $\rightarrow$ peak bin is $k = 1$
- while bin k holds the maximum energy peak, energy still leaks into adjacent bins due to windowing (spectral leakage)
- DFT bins are **point samples**, **NOT** a frequency range or “intervals”

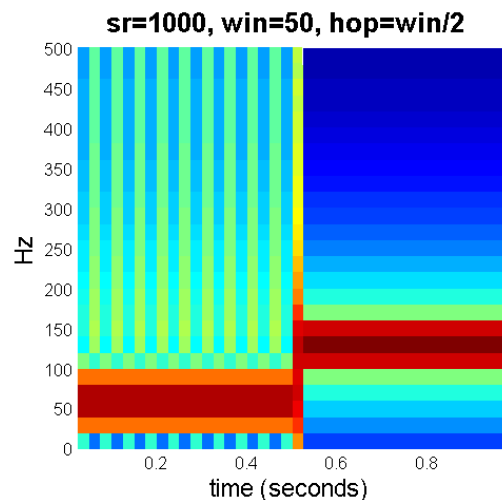
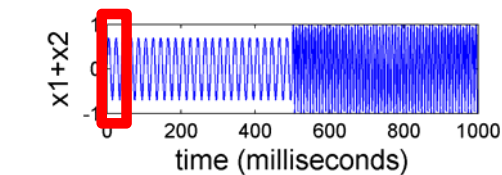


Trade-off between Frequency/Temporal Resolution in STFT

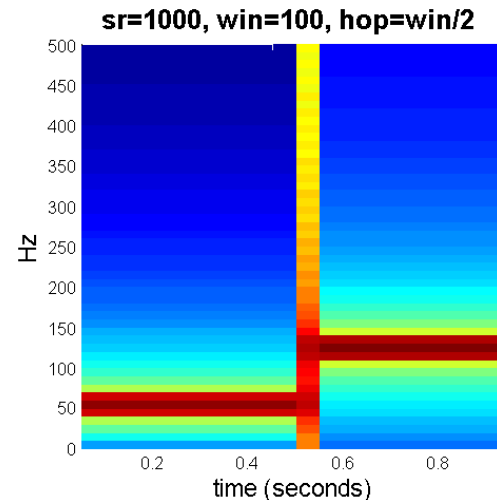
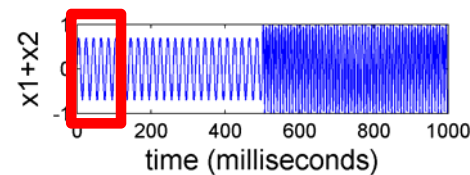
- **Frequency resolution:** $\frac{F_s}{N}$
 - Longer window size (N) → finer frequency resolution (but larger resulting STFT)
→ can localize events along the frequency axis
- **Temporal resolution:** $\frac{H}{F_s}$
 - Smaller hop size (H) → finer temporal resolution (but larger resulting STFT)
→ can localize events along the time axis
- If $H = N/R$ (e.g., $R = 4$)
 - Temporal resolution: $\frac{N}{RF_s}$ (no good freq/time resolution at the same time)

Trade-off between Frequency/Temporal Resolution in STFT

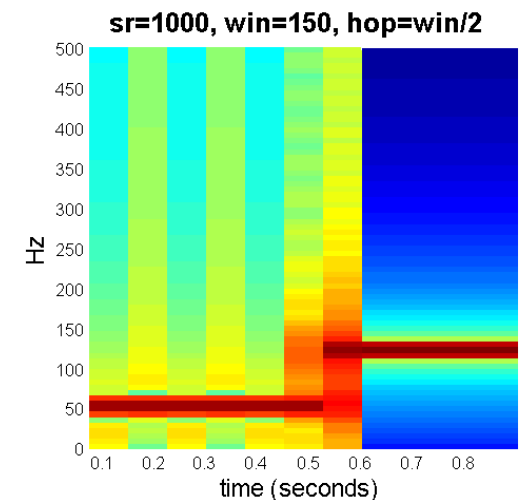
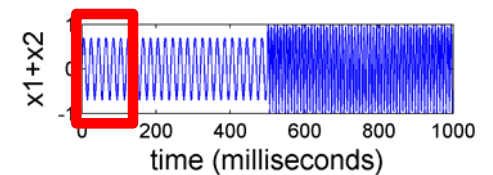
- Different window size ($\text{win_size} = 50, 100, 150$)
- $\text{hop_size} = \text{win_size}/2$



size: 26 x 39



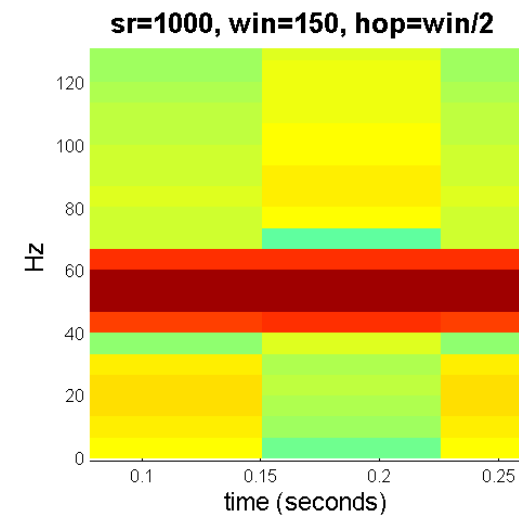
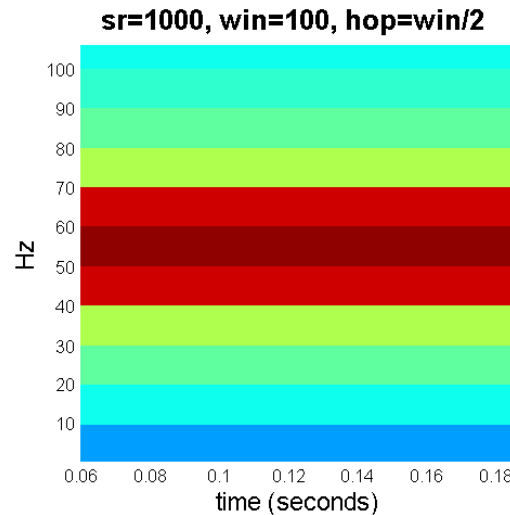
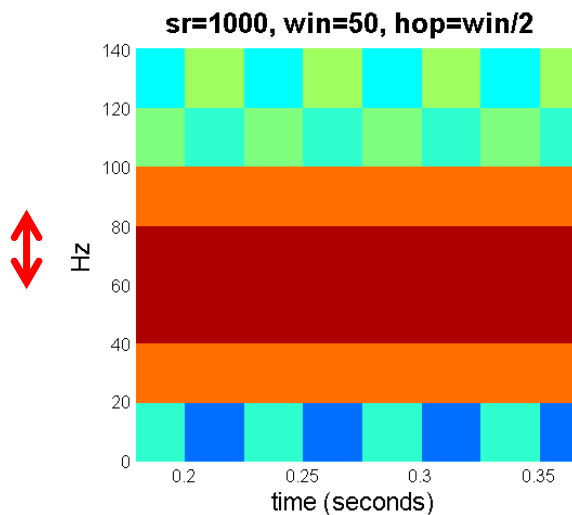
size: 51 x 19



size: 76 x 12

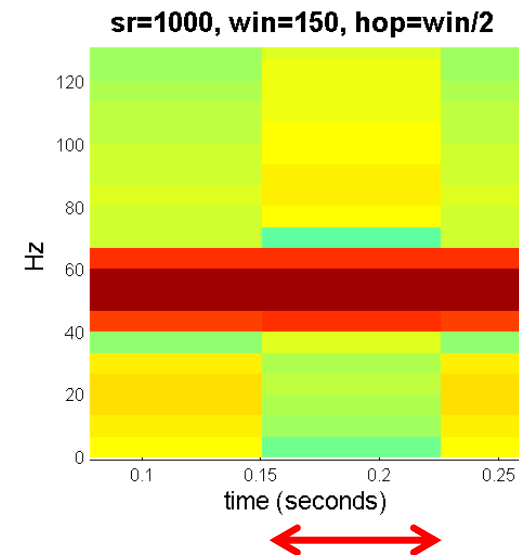
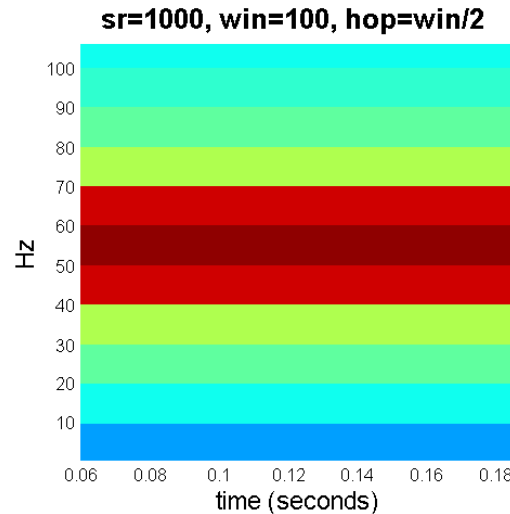
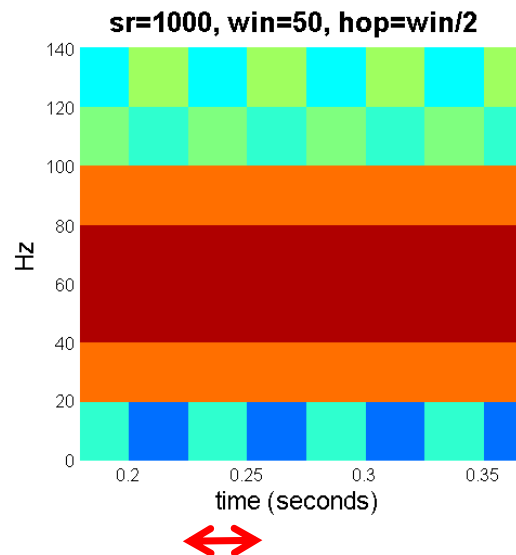
Frequency Resolution in STFT

- **freq_resolution = $F_s/\text{win_size}$**
 - longer window $\rightarrow$ better frequency resolution
 - freq_resolution = 20, 10, 6.6667 (Hz), respectively



Understanding STFT

- **temporal_resolution: $\text{hop_size}/F_s$**
 - longer hop $\rightarrow$ worse temporal resolution
 - temp_resolution = 25, 50, 75 (ms), respectively



torchaudio.transforms.Spectrogram

<https://docs.pytorch.org/audio/main/generated/torchaudio.transforms.Spectrogram.html>

```
waveform, sample_rate = torchaudio.load("test.wav", normalize=True)
transform = torchaudio.transforms.Spectrogram(n_fft=800)
spectrogram = transform(waveform)
```

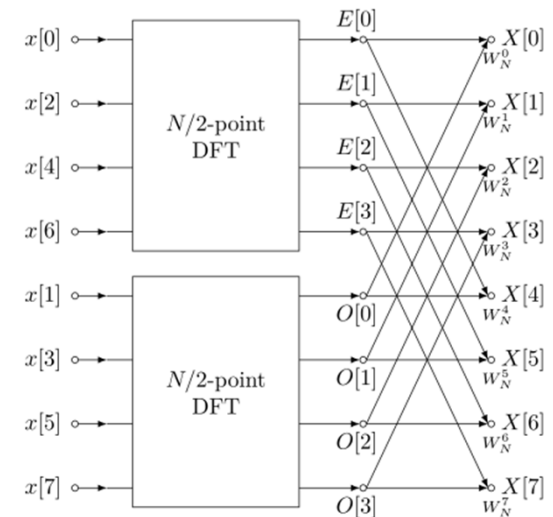
Parameters

- **n_fft** (*int, optional*) – Size of FFT, creates $n_fft // 2 + 1$ bins. (Default: 400)
- **win_length** (*int or None, optional*) – Window size. (Default: `n_fft`)
- **hop_length** (*int or None, optional*) – Length of hop between STFT windows. (Default: $win_length // 2$)

DFT, STFT, FFT (Fast Fourier Transform)

https://www.audiolabs-erlangen.de/resources/MIR/FMP/C2/C2_DFT-FFT.html

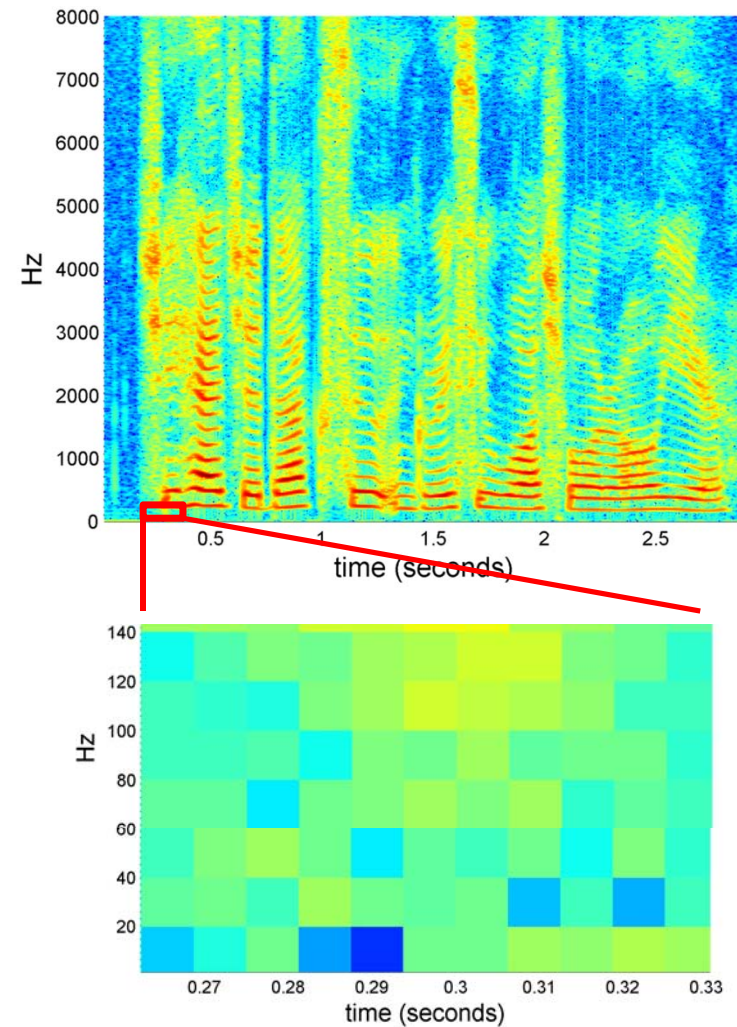
- **FFT** is a fast algorithm to compute the DFT; the Cooley-Tukey algorithm requires an array length that is an exact power of 2 (e.g., 512, 1024)
- In practice, we set $N_w = N_{fft}$
- But they can be decoupled
 - N_w : physics of the measurement (determines physical resolution)
 - N_{fft} : computation and representation (grid density/interpolation)
 - $N_w \leq N_{fft}$ (via zero-padding)



https://en.wikipedia.org/wiki/Fast_Fourier_transform

Quiz

1. The figure on the top-right is the spectrogram of a signal. What is the underlying sampling rate?
2. The figure on the bottom-right is a zoom-in of the above figure. We can see that the frequency resolution is 20 Hz. What is the window size (in samples)?
3. The temporal resolution is close to 6.6 ms. What's the hop size (in samples), approximately?



Quiz

4. Given an EEG headset that samples signals at 128 Hz, if we want to be able to discriminate frequency components that differ by 0.5 Hz in frequency, what is the minimal window size (in samples) we need to use? What is the length of such a window in seconds then?
5. Following the previous question, if we further want to discriminate events that differ in time by 0.5 second, what is the maximal hop size (in samples) we need to use?

Brainwaves, Frequencies and Functions

Unconscious		Conscious		
Delta	Theta	Alpha	Beta	Gamma
0,5 – 4 Hz	4 – 8 Hz	8 – 13 Hz	13 – 30 Hz	30-42 Hz
Instinct	Emotion	Consciousness	Thought	Will

Quiz

6. Given a music signal with $sr = 44,100$ Hz, when we use a window size of 1,024 samples, what would be the frequency resolution?
7. According to the figure on the right, we know that the fundamental frequency (f_0) of A1 is 55 Hz. Following the previous question, which bin would be A1's peak bin ?

Note name	Keyboard	<u>Frequency</u> Hz	
A0		27.500	
B0		30.868	29.135
C1		32.703	
D1		36.708	34.648
E1		41.203	38.891
F1		43.654	
G1		48.999	46.249
A1		55.000	51.913
B1		61.735	58.270
C2		65.406	
D2		73.416	69.296
E2		82.407	77.782
F2		87.307	
G2		97.999	92.499
A2		110.00	103.83
B2		123.47	116.54
C3		130.81	
D3		146.83	138.59
E3		164.81	155.56
F3		174.61	
G3		196.00	185.00
A3		220.00	207.65
B3		246.94	233.08
C4		261.63	
D4		293.67	277.18
		311.13	

Quiz

6. Given a music signal with $sr = 44,100$ Hz, when we use a window size of 1,024 samples, what would be the frequency resolution?

Sol: 43.1 Hz

7. According to the figure on the right, we know that the fundamental frequency (f_0) of A1 is 55 Hz. Following the previous question, which bin would be A1's peak bin ?

Sol: $k = 1$

Note name	Keyboard	Frequency Hz		
A0		27.500		0
B0		30.868	29.135	
C1		32.703		
D1		36.708	34.648	
E1		41.203	38.891	43.1
F1		43.654		
G1		48.999	46.249	
A1		55.000	51.913	
B1		61.735	58.270	
C2		65.406		
D2		73.416	69.296	
E2		82.407	77.782	86.2
F2		87.307		
G2		97.999	92.499	
A2		110.00	103.83	
B2		123.47	116.54	129.3
C3		130.81		
D3		146.83	138.59	
E3		164.81	155.56	172.4
F3		174.61		
G3		196.00	185.00	
A3		220.00	207.65	215.5
B3		246.94	233.08	
C4		261.63		
D4		293.67	277.18	
		329.63	311.13	

Quiz

8. Following the `6` and `7`;
how if we use a window size of 4,096 samples?

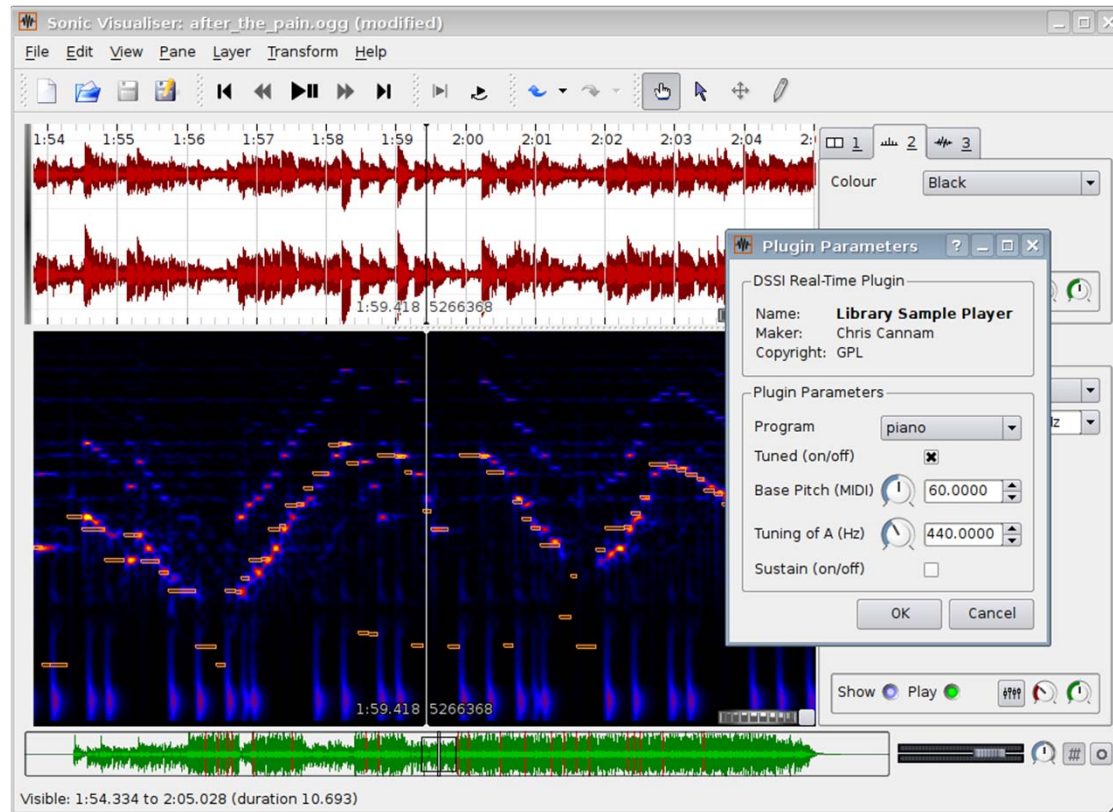
Note name	Keyboard	Frequency Hz	
A0		27.500	
B0		30.868	29.135
C1		32.703	
D1		36.708	34.648
E1		41.203	38.891
F1		43.654	
G1		48.999	46.249
A1		55.000	51.913
B1		61.735	58.270
C2		65.406	
D2		73.416	69.296
E2		82.407	77.782
F2		87.307	
G2		97.999	92.499
A2		110.00	103.83
B2		123.47	116.54
C3		130.81	
D3		146.83	138.59
E3		164.81	155.56
F3		174.61	
G3		196.00	185.00
A3		220.00	207.65
B3		246.94	233.08
C4		261.63	
D4		293.67	277.18
		311.13	

Summary

- **Sampling rate, window size, hop size** all matter
 - Frequency resolution
 - Temporal resolution
 - Number of samples, number of frames, and physical time (in milliseconds)
- Make sure the sampling rate is “right”
 - Do “resample” if the sampling rate of your audio files is different from what is assumed by an open-source model
 - Speech: 16k Hz
 - Music: 22k, 44k, or 48k Hz
 - Similarly for window size, hop size, window function, etc

Appendix: Sonic Visualiser

<http://www.sonicvisualiser.org/>



Appendix: Audacity

<https://www.audacityteam.org/>

